School of Computer Science and Electronic Engineering

Practical Business Analytics (COMM053) Coursework

Academic Year 2025–26

[Group FEB G01]

A project report submitted by:

Lohit (ID: 6647300)
Nneamaka (ID: 6901085)
Upasana (ID: 6902185)
Shivam (ID: 6903655)
Amarjeet (ID: 6908357)

University of Surrey
School of Computer Science and Electronic Engineering
Guildford, Surrey GU2 7XH

1 Introduction (Business Understanding)

1.1 Business Understanding

Online commerce has grown rapidly, bringing significant opportunities for businesses but also exposing them to increasingly sophisticated fraud. Fraudulent transactions lead to direct financial losses, chargebacks, penalties from payment processors, reputational damage, and customer churn. At the same time, overly aggressive fraud prevention mechanisms that incorrectly block legitimate transactions can negatively impact customer experience and revenue. Therefore, businesses must strike a careful balance between detecting fraud effectively and maintaining smooth, trustworthy customer interactions. The core business problem addressed in this project is the accurate and timely identification of fraudulent e-commerce transactions. For each incoming transaction, the system must decide whether it is fraudulent or legitimate quickly enough to support near real-time decision-making. Fraud arises from factors such as stolen payment credentials, account takeovers, synthetic identities, and automated attacks exploiting weak verification processes. Failure to detect these activities results in financial and operational consequences, while false alarms increase manual review costs and frustrate genuine customers. From a business analytics perspective, fraud detection is a high-impact application where data-driven decision support can significantly reduce risk while preserving customer trust.

1.2 Problem Statement and Objectives

The problem addressed in this project is to identify fraudulent e-commerce transactions at the point of purchase using historical transaction records. We model this as a supervised binary classification task, where the target variable `is_fraud` equals 1 for fraudulent transactions and 0 for legitimate transactions.

The key business objectives are:

- Minimise false negatives (missed fraud) to reduce direct financial losses and chargebacks.
- Control false positives to avoid unnecessary transaction declines, reduce manual review load, and limit customer dissatisfaction.
- Develop scalable models suitable for high-volume, low-latency transaction environments.
- Provide interpretable insights to support risk analysts, management decisions, and regulatory accountability.

These objectives reflect the real-world trade-offs faced by online merchants and payment providers.

1.3 Research Question

Which supervised learning approach (individual models vs. stacking/ensemble approaches) yields the best fraud detection performance on an imbalanced e-commerce dataset?

1.4 Dataset Description

Data source: E-Commerce Fraud Detection Dataset [8].
The dataset consists of transaction-level e-commerce payment records, where each row represents an individual online transaction attempt. It contains approximately 300,000 transactions generated by around 6,000 unique users, with each user performing multiple transactions on average (approximately 40–60 per user). This structure enables meaningful behavioural analysis, allowing the modelling process to capture patterns across repeated user activity rather

than relying solely on isolated transactions. The data includes a rich set of behavioural, transactional, geographic, and security-related features, such as transaction amount, user account age, and transaction history, payment verification outcomes, merchant category, channel, and shipping distance. The target variable `is_fraud` indicates whether a transaction is fraudulent (1) or legitimate (0) The dataset contains a mix of:

- **Numerical features** (e.g., amount, account_age_days, shipping_distance_km)
- **Categorical features** (e.g., country, merchant_category, channel)
- **Binary indicators** (e.g., promo_used, avs_match, three_ds_flag)

Fraudulent transactions account for only approximately 2% of all observations, resulting in a highly imbalanced dataset. This reflects real-world fraud detection scenarios, where fraudulent behaviour is rare but costly, and it necessitates careful model selection, appropriate evaluation metrics, and decision strategies that prioritise the detection of minority-class events. Overall, the dataset closely mirrors operational e-commerce payment systems by combining repeated user behaviour, contextual transaction information, and security signals. Its size, structure, and imbalance make it well-suited to evaluating supervised learning and ensemble-based fraud detection approaches in a realistic business analytics setting.

1.5 Planned Analytical Techniques

To address the research question(s), the project applies supervised machine learning techniques for binary classification. As a group, a common baseline stacking-based ensemble is constructed using Logistic Regression, K-Nearest Neighbours, Random Forest, and Gaussian-based models. Individual group members explore different aggregation strategies to assess how decision-level combinations impact performance and business trade-offs. In addition, each group member independently implements a second supervised learning model to explore alternative modelling paradigms, such as margin-based, tree-based, linear generative, or neural network-based approaches. Model performance is evaluated using consistent metrics, including recall, precision, F1-score, and ROC-AUC, with particular emphasis on recall due to the high cost of undetected fraud.

1.6 CRISP-DM Methodology

This project follows the CRISP-DM framework:

1. **Business Understanding:** We define the fraud detection objective, costs of false negatives vs. false positives, and the operational requirement for timely decision-making.
2. **Data Understanding:** We inspect feature types, class imbalance, and basic distributions to understand the dataset in the context of fraud risk signals.
3. **Data Preparation:** It involves cleaning and transforming the data for modelling (e.g., encoding categorical variables, scaling numerical features where required, and finalising an input feature set that is valid at prediction time).
4. **Modelling:** We train baseline supervised models and a stacking ensemble and compare alternative aggregation strategies.
5. **Evaluation:** We compare models using consistent metrics (with emphasis on recall).
6. **Deployment (discussion):** We interpret the model output as a risk score to support fraud screening decisions (e.g., approve, step-up authentication, or manual review), and present business-oriented recommendations, focusing on fraud prevention and customer experience.

2 Data Understanding, Pre-Processing and Exploration

2.1 Data Inspection and Quality Assessment

This section describes the data inspection and cleaning steps performed to ensure the transactional dataset was structurally valid, consistent, and suitable for subsequent analysis. All procedures were implemented using the `R` programming environment.

2.1.1 Environment Setup and Data Ingestion

Code Reference: `library(tidyverse)`, `read.csv(...)`

To ensure efficient data manipulation and visualization, the analysis was conducted using the `R` programming environment with the Tidyverse suite of packages. The raw transactional dataset (`transactions.csv`) was ingested into the system. A robust loading mechanism was implemented to verify file path existence, ensuring reproducibility across different team members' local environments.

2.1.2 Variable Type Conversion

Code Reference: `as.factor(...)`

The raw dataset initially interpreted several categorical variables as integers or strings. To ensure statistical accuracy, selected variables were explicitly encoded as categorical factors. The target variable `is_fraud` was converted into a binary classification factor distinguishing legitimate and fraudulent transactions. Transactional context variables such as `merchant_category` and `channel` were converted to nominal factors, along with the security indicator `avs_match`, were also converted to categorical factors to enable appropriate group-based analysis.

2.1.3 Temporal Parsing

Code Reference: `as.POSIXct(..., format = "%Y-%m-%dT%H:%M:%SZ")`

The `transaction_time` variable was originally provided in ISO 8601 string format. This feature was parsed into POSIXct date-time objects to enable validation of the dataset's temporal scope and to support time-based exploratory analysis, such as examining transaction activity across different periods.

2.1.4 Data Quality Assurance

Code Reference: `is.na`, `duplicated`, `range`, `amount < 0`

Prior to exploratory data analysis, a comprehensive data quality assessment was conducted. The dataset was examined for missing values, and no missing observations were detected. Duplicate records were checked to prevent skewed frequency counts, with no duplicates identified. Logical validity checks were applied to the `amount` variable to ensure no negative transaction values existed. Additionally, temporal consistency was verified, confirming that the dataset spans from **January 1, 2024** to **October 31, 2024**.

2.1.5 Feature Selection

Code Reference: `select(-transaction_id, -user_id)`

Following data validation, a feature selection step was performed to optimize the dataset for analysis. The variables `transaction_id` and `user_id` were removed, as they are high-cardinality unique identifiers that do not contain intrinsic predictive information regarding fraud behaviour. Their removal reduces noise and prevents downstream models from memorizing identifier values instead of learning generalizable behavioural patterns.

2.1.6 Summary of Data Readiness

The data preparation process resulted in a structured and validated dataset consisting of **299,695 observations** and **15 behavioural and contextual features**. No missing values, duplicate records, or logical inconsistencies were identified. The dataset is therefore considered suitable for subsequent exploratory data analysis and machine learning modelling.

2.2 Exploratory Data Analysis (EDA)

2.2.1 Class Imbalance Analysis

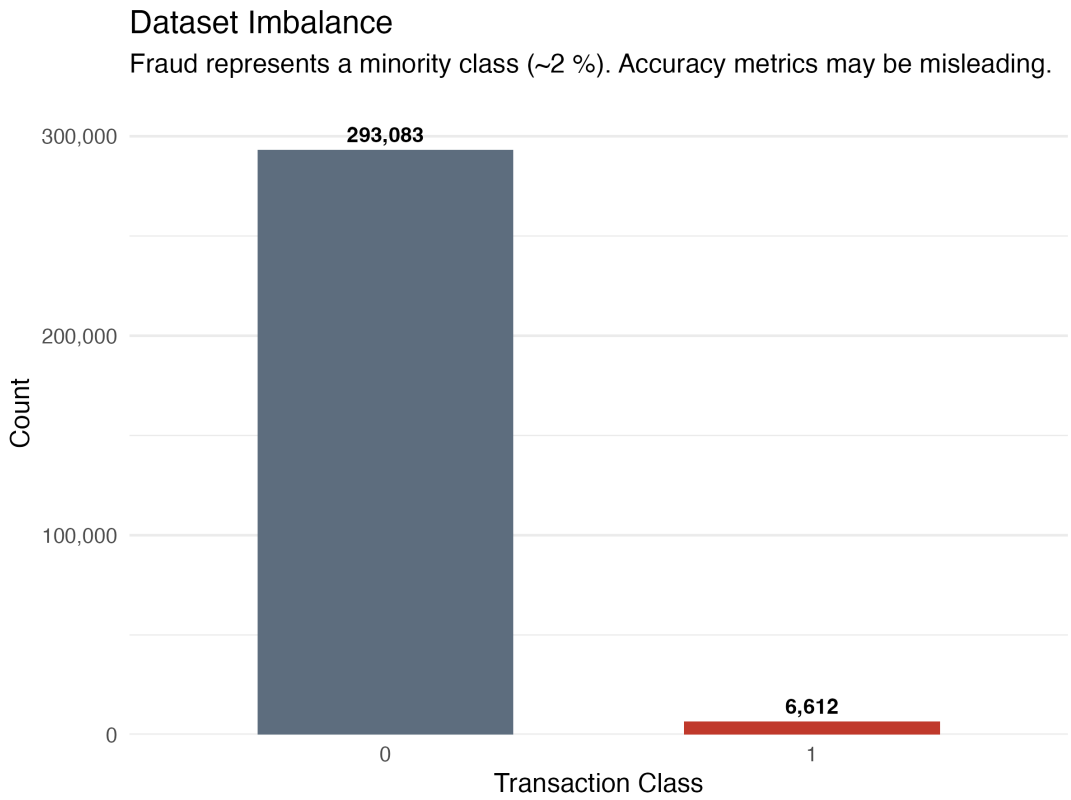


Figure 1: Class Distribution.

In Figure 1 The class distribution analysis reveals a highly imbalanced dataset, in which fraudulent transactions constitute only a small minority of observations (approximately 2%), while the vast majority represent legitimate activity. This imbalance reflects the operational reality of real-world fraud detection and carries important business implications. In such settings, conventional performance measures such as overall accuracy can be misleading, as a model that predicts all transactions as non-fraudulent would still appear to perform well despite providing no meaningful fraud prevention value. From a business perspective, the low frequency of fraud combined with its potentially significant financial and reputational impact necessitates a risk-focused analytical approach. Accordingly, this imbalance justifies the use of evaluation metrics that prioritise fraud identification capability, including recall, precision, and area-based measures, and informs model design choices aimed at maximising fraud detection while minimising unnecessary disruption to legitimate customers

2.2.2 Spending Behaviour Patterns and Fraud Risk

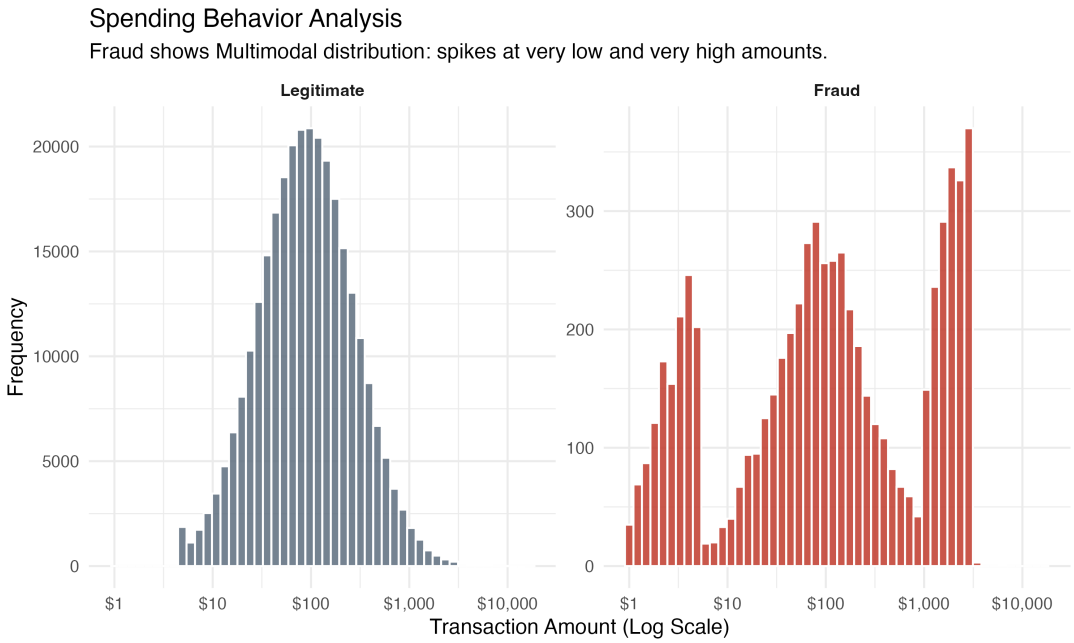


Figure 2: Spending Behavior

The spending behaviour analysis reveals structurally different transaction value patterns between legitimate and fraudulent activity. Legitimate transactions are largely concentrated within a single mid-range cluster, with the highest frequency observed approximately between $20 and $300, indicating consistent customer spending behaviour. In contrast, fraudulent transactions display a multi-modal distribution with three discernible clusters. A first cluster appears at very low transaction values (below $10), consistent with testing or probing behaviour. A second cluster is observed at mid-range values (approximately $50 to $300), overlapping with legitimate transactions and therefore posing a higher detection challenge. A third cluster occurs at high transaction values, typically exceeding $1,000, reflecting attempts to maximise financial gain following successful account compromise. From a business perspective, these patterns suggest two dominant fraud strategies: low-value transactions associated with card testing and high-value transactions aimed at rapid value extraction. This spending asymmetry reinforces the need for fraud detection models capable of identifying risk across both low- and high-value transactions, ensuring that high-impact losses are prevented while legitimate mid-range customer spending remains largely unaffected

2.2.3 Shipping Distance Risk Analysis

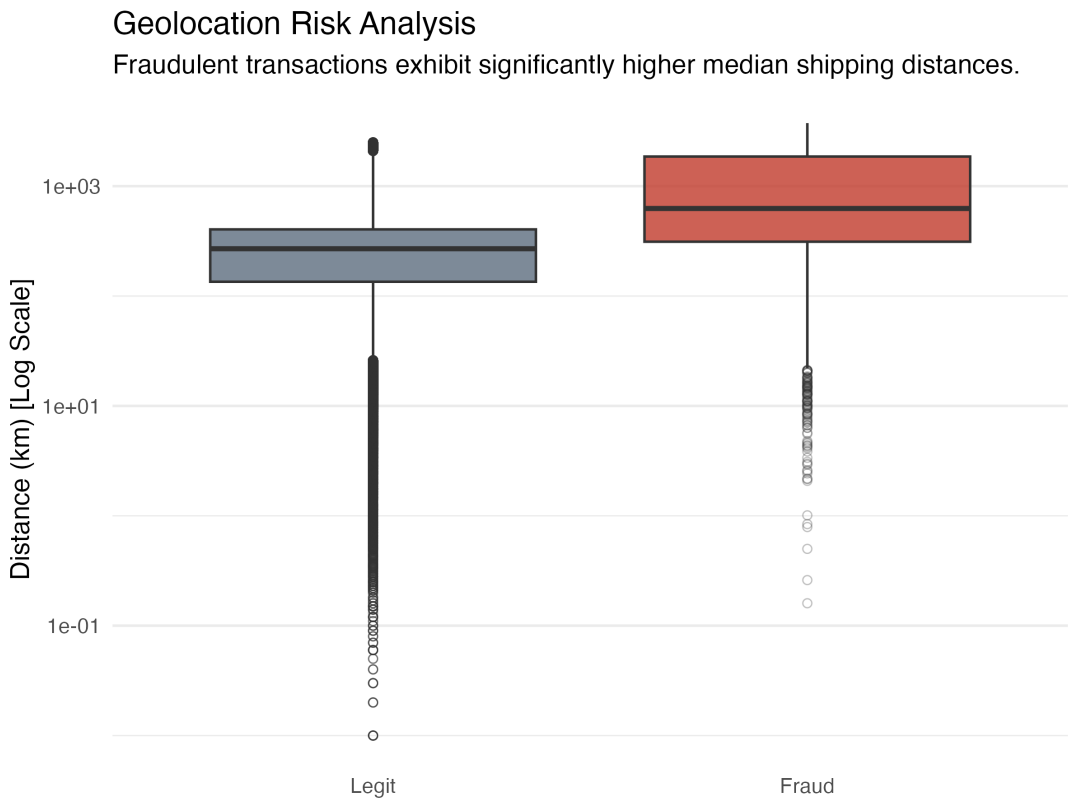


Figure 3: Shipping Distance Risk Analysis

Figure 3 reveals a clear and material separation between legitimate and fraudulent transactions in terms of shipping distance, as shown on the logarithmic scale. Legitimate transactions are typically associated with shorter shipping distances, with the median distance appearing to lie in the range of approximately 50–150 km, and the majority of transactions concentrated below 500 km, reflecting stable and localised customer purchasing behaviour. In contrast, fraudulent transactions exhibit a substantially higher median shipping distance, with the median positioned closer to 800–1,000 km and a broader interquartile range extending beyond this level. Additionally, fraudulent transactions show a greater spread of long-distance outliers, frequently exceeding 1,000 km, indicating cross-regional or cross-border delivery patterns. From a business perspective, this spatial divergence suggests that fraud is more likely to involve geographically anomalous shipping locations, increasing operational risk through higher delivery costs, delayed detection, and elevated chargeback exposure. Consequently, these findings support the inclusion of distance-based geolocation features in fraud detection models and justify the use of risk controls that flag unusually distant transactions while minimising unnecessary friction for legitimate customers with consistent delivery behaviour

2.2.4 Risky Merchant Sectors

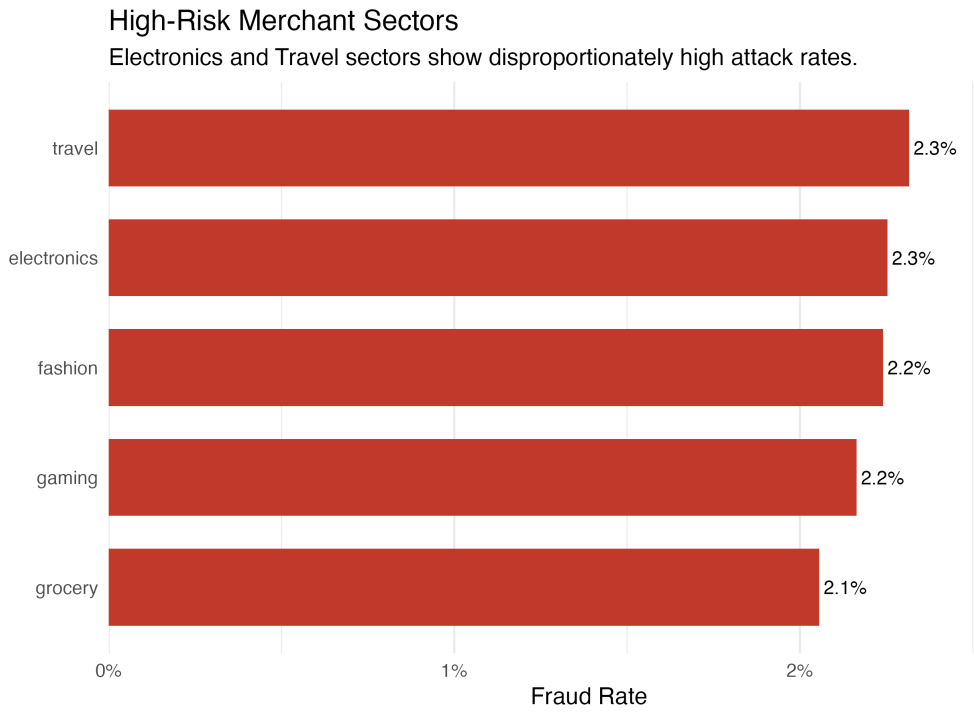


Figure 4: Top 5 Risky Merchant Sectors

Figure 4 reveals meaningful variation in risk exposure by transaction category. As shown in the chart, the Travel and Electronics sectors exhibit the highest fraud rates, each at approximately 2.3%, indicating a disproportionately higher susceptibility to fraudulent activity relative to other sectors. Fashion and Gaming follow closely, both with fraud rates around 2.2%, while Grocery transactions show a comparatively lower, though still material, fraud rate of approximately 2.1%. From a business perspective, these differences suggest that fraud risk is not uniformly distributed across merchant categories but is concentrated in sectors associated with high-value goods, digital delivery, or limited post-transaction verification. This sector-level asymmetry highlights the importance of incorporating merchant category information into fraud detection models and supports the adoption of differentiated risk controls, enabling enhanced monitoring for high-risk sectors.

2.2.5 Temporal Transaction Patterns

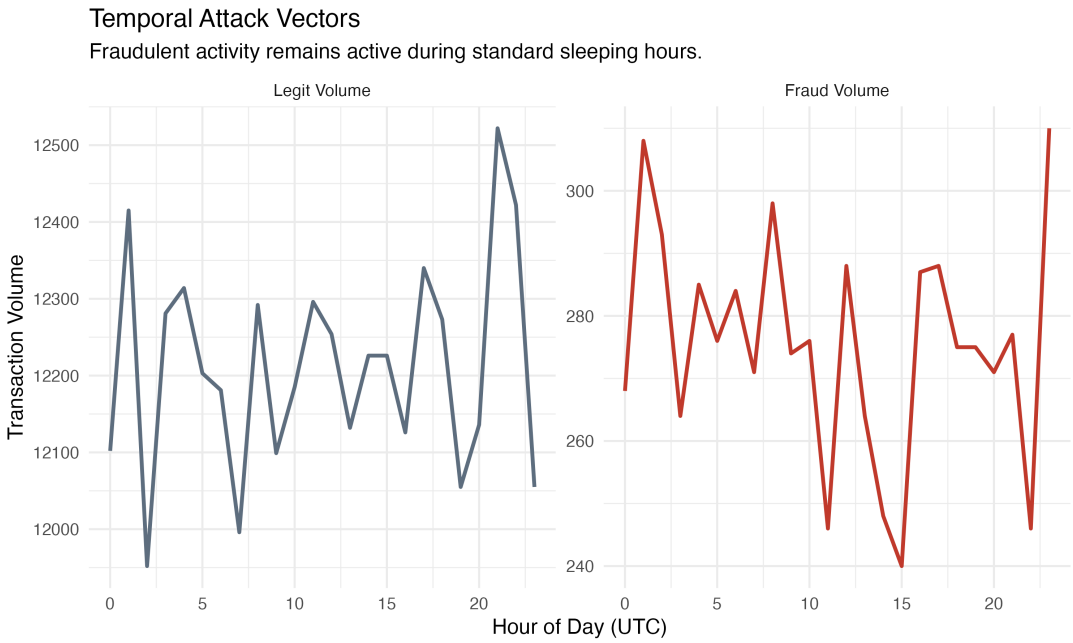


Figure 5: Temporal Transaction Patterns

In Figure 5 the temporal analysis of transaction volumes shows distinct behavioural differences between legitimate and fraudulent activity across the 24-hour cycle. Legitimate transaction volume remains relatively stable throughout the day, fluctuating narrowly between approximately 12,000 and 12,500 transactions per hour, with modest increases during typical active hours and no pronounced overnight decline. In contrast, fraudulent transaction volume is consistently present across all hours, ranging roughly between 240 and 310 transactions per hour, and does not exhibit a clear reduction during standard sleeping hours (approximately 00:00–06:00 UTC). Notably, fraud volumes remain elevated during late-night and early-morning periods especially between 2am to 5am, while legitimate activity shows less variability. From a business perspective, this temporal persistence suggests that fraudulent activity is less constrained by human behavioural cycles and is likely driven by automated or coordinated processes. This insight underscores the importance of continuous, round-the-clock fraud monitoring and supports the inclusion of time-based features in fraud detection models, enabling institutions to identify elevated risk during periods when legitimate customer activity is relatively stable but fraud remains active.

2.2.6 Address Verification Security (AVS) Risk

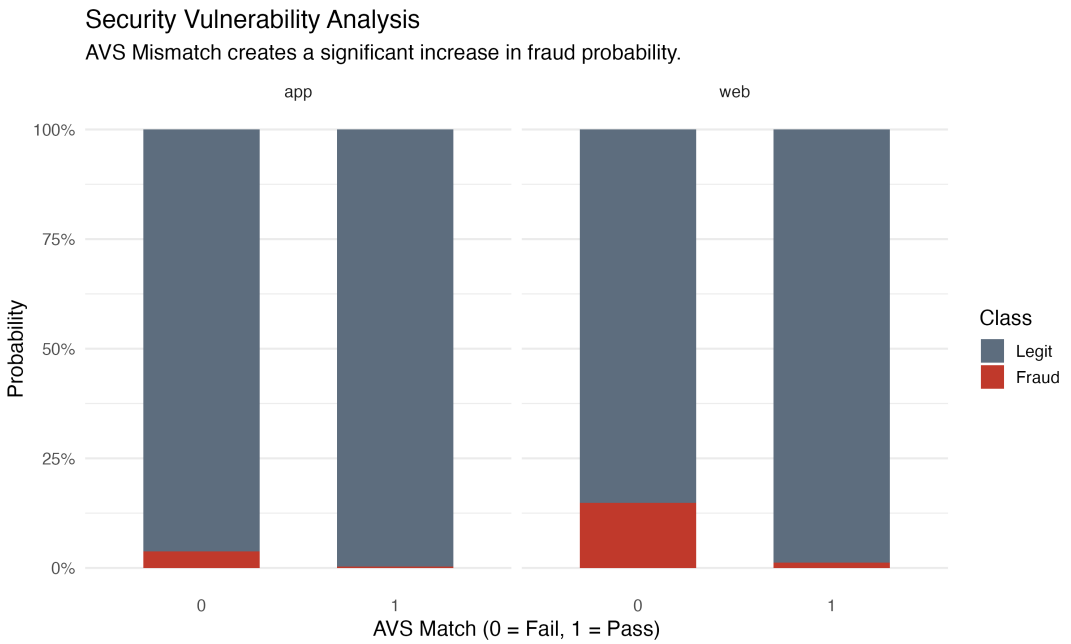


Figure 6: AVS Mismatch Analysis

The Figure 6 conveys a clear and measurable increase in fraud risk when address verification fails, across both app and web channels. For web transactions, failed AVS checks (AVS = 0) are associated with a fraud rate of approximately 10–15%, compared to around 1–2% when the address verification passes. This represents a substantial increase in risk linked directly to AVS mismatch. In the app channel, the same pattern is present but less pronounced, with failed AVS transactions showing a fraud rate of roughly 3–4%, compared to close to 0–1% when AVS passes. From a business perspective, these figures indicate that AVS mismatch is a strong and quantifiable risk signal, particularly for web-based purchases, and can be used to clearly distinguish higher-risk transactions from the majority of low-risk, verified activity.

2.3 Fraud Rate per Country

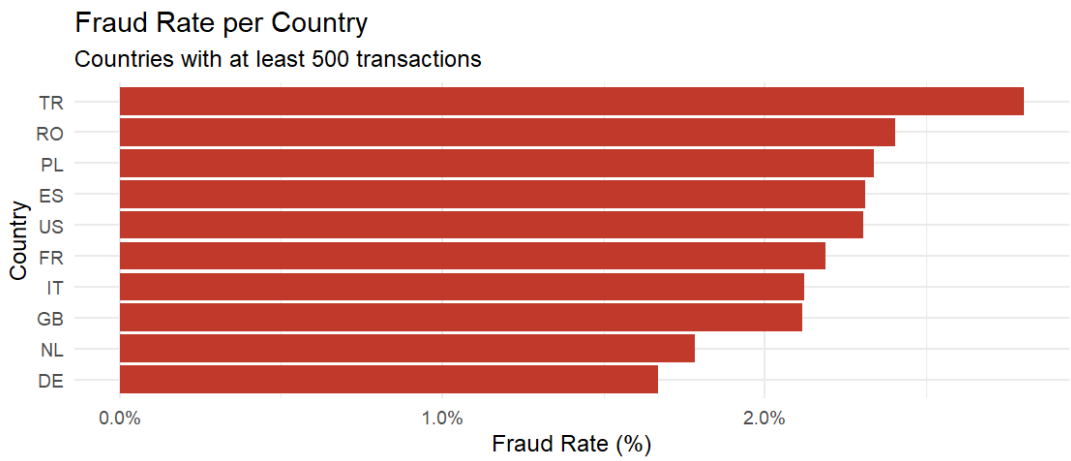


Figure 7: Fraud Rate per Country

Figure 7 above shows the rate of fraud that occurs in each country. It is calculated as the proportion of fraudulent transactions relative to the total number of transactions. To avoid unstable estimates and ensure reliable comparisons, it does not include countries with less than 500 transactions. Focusing on fraud rates and not the raw transaction counts allows for a much fairer comparison between countries regardless of how many transactions they process. The results show that the risk of fraud is not equally distributed in each country. Some countries have much higher fraud rates than others even when the amount of transactions they process are similar. This shows that location is an important factor in detecting and understanding fraudulent behaviour. Fraud risk varies across countries, not just because some countries have higher fraudulent transactions, but because the likelihood of the fraud itself is different from time to time. Fraud risks vary due to differences in how payments are handled and how fraud is managed locally. This justifies the use of the country-related features in the modeling stage. This insight is valuable to the fraud risk analysis industry because it points out situations where additional monitoring, stricter controls, or region-specific fraud rules may be necessary.

2.4 Distribution and Skewness of Key Transaction Variables

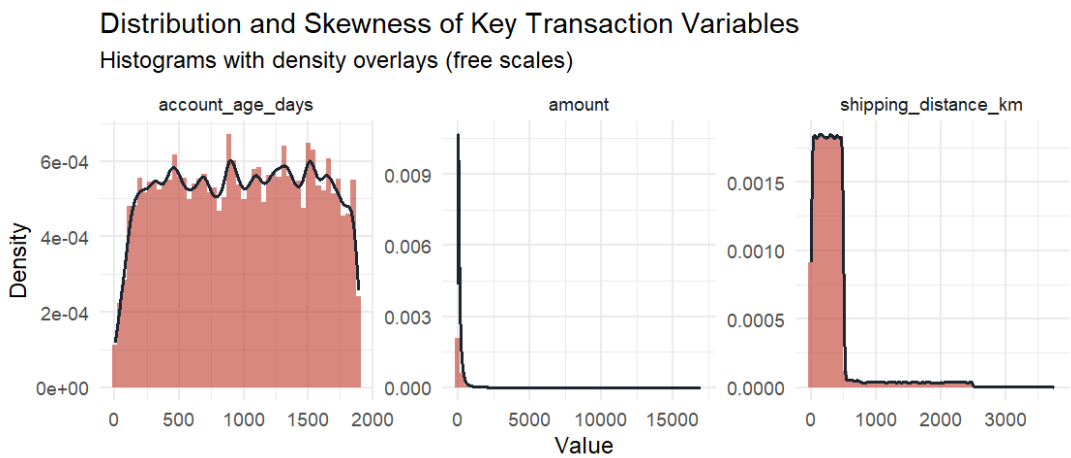


Figure 8: Distribution and Skewness of Key Transaction Variables

Figure 8 shows how the selected key transaction variables are distributed across the dataset. It highlights the presence of right skewed distributions across the amount of transactions, shipping distance, and account age. The distributions are uneven such that most of the transactions have small values, while the remaining few transactions are very large. This is a common pattern which is often associated with unusual risk behaviour in real transaction data. It is important in fraud detection because fraudulent transactions are usually very different from normal customer behaviour. They could involve large transactions, longer shopping distances, or any other suspicious patterns. Because of this, it is necessary to scale the data and work with models that can work effectively with extreme values. This justifies the use of ensemble models as they combine multiple predictions and reduce the impact of outliers and irregular transaction patterns. In conclusion, this analysis shows that the data used for modelling reflects the behaviour of real world transactions.

2.5 Feature Engineering and Preparation

The feature engineering phase aimed to transform raw transactional attributes into informative, model-ready predictors by capturing behavioural patterns, temporal context, and transaction characteristics relevant to fraud detection. Rather than relying on raw timestamps and categorical variables, new features were constructed to represent *when* a transaction occurred, *under*

what conditions, and *how unusual* the context was compared to typical customer behaviour. All transformations were implemented using a reproducible pipeline in the `R` programming environment.

2.5.1 Numerical Feature Assessment and Reduction

Code Reference: Numerical feature selection using `select(where(is.numeric))`, correlation computation via `cor()`, and heatmap visualisation using `ggplot()` with `geom_tile()`. Feature removal performed using `select(-avg_amount_user, -promo_used)`.

A preliminary assessment of numerical features was conducted using correlation analysis to examine linear relationships and identify potential redundancy. Based on this analysis, the features `avg_amount_user` and `promo_used` were removed. These variables did not provide additional predictive value beyond existing behavioural and transactional attributes and risked introducing noise or multicollinearity. Removing them reduced dimensionality while preserving meaningful information for fraud detection.

2.5.2 Temporal Feature Engineering

Code Reference: Timestamp parsing and standardisation using `ymd_hms(transaction_time, tz = "UTC")`, removal of invalid timestamps using `filter(!is.na(transaction_time))`, and extraction of temporal components using `hour()`, `day()`, `month()`, and `wday(week_start = 1)`.

The original transaction timestamp (`transaction_time`) was parsed and standardised to Coordinated Universal Time (UTC) to ensure temporal consistency across records. Transactions with invalid timestamps were removed to maintain data integrity. From the cleaned timestamp, raw temporal components were extracted, including hour of day, day of month, month of year, and weekday.

These features represent the temporal context of each transaction and enable the model to capture time-dependent behavioural patterns, such as differences between daytime and night-time activity or weekday and weekend behaviour, which are commonly associated with changes in fraud risk.

2.5.3 Behavioural Context Encoding

Code Reference: Binary feature creation using conditional logic via `ifelse()`, and channel encoding using `rename()` after binary transformation.

To translate raw temporal and categorical information into interpretable behavioural signals, binary indicator features were constructed. The feature `evening` was derived from the hour of transaction and flags activity occurring outside standard business hours. The feature `weekend` was derived from the weekday variable and identifies transactions occurring on Saturdays or Sundays. In addition, the original `channel` variable was encoded into a binary representation distinguishing web-based from app-based transactions.

These features capture simplified behavioural contexts that allow models to distinguish between typical and atypical transaction conditions using clear, interpretable signals.

2.5.4 Cyclical Representation of Time

Code Reference: Cyclical transformations implemented using `sin()` and `cos()`, with cycle normalisation using `days_in_month()` for intra-month scaling.

Temporal variables are inherently cyclical; for example, hour 23 and hour 0 are adjacent in time despite being numerically distant. To preserve this structure, sine and cosine transformations were applied to the extracted temporal components. Specifically, cyclical encodings were created for hour of day, weekday, day of month, and month of year.

These transformations allow machine learning models to learn periodic patterns and temporal proximity correctly, improving their ability to detect fraud patterns linked to daily, weekly, or seasonal cycles.

2.5.5 Final Feature Consolidation

Code Reference: Redundant variable removal using `select()`, dataset export using `write.csv()`, and stratified train–test splitting using `createDataPartition()` with `set.seed()` for reproducibility.

After feature construction, original raw variables that were replaced by engineered representations, including the original timestamp and intermediate temporal components were removed to avoid redundancy. The final dataset therefore consists exclusively of numerical and binary features suitable for supervised learning algorithms. The engineered data was exported and split into training and testing sets using a reproducible stratified 80/20 split to support robust model evaluation.

2.5.6 Summary

Overall, feature engineering transformed raw timestamps and categorical attributes into behavioural and cyclical predictors that explicitly capture transaction timing and context. These engineered features enhance the model's ability to distinguish between normal customer behaviour and anomalous transaction patterns associated with fraud, while ensuring numerical compatibility, interpretability, and modelling robustness.

3 Modelling

3.1 Baseline Model Structure

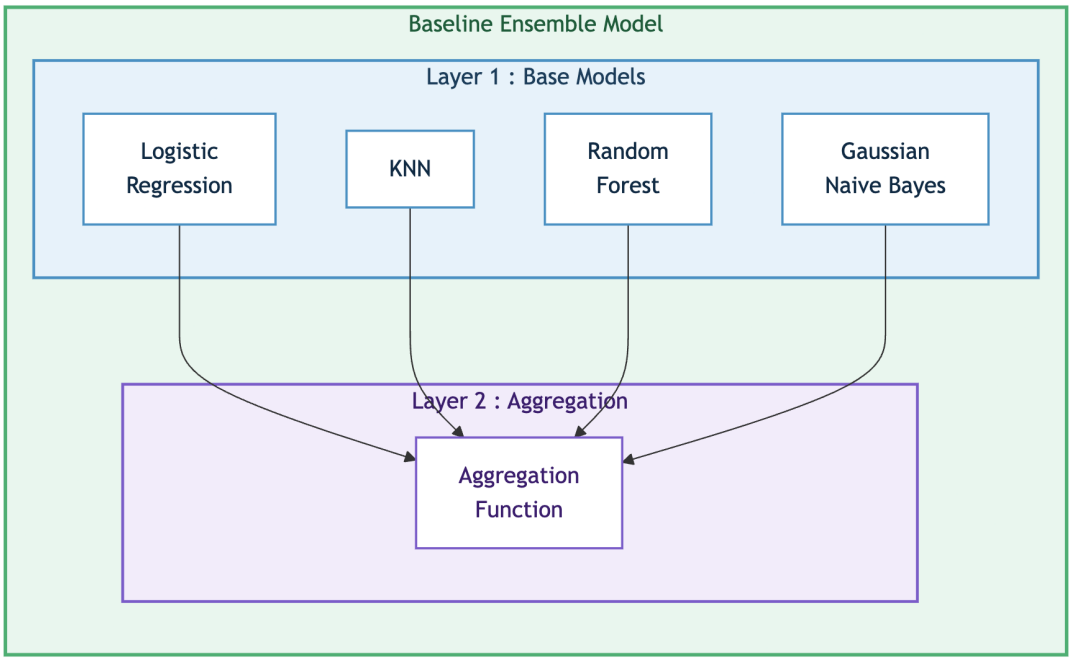


Figure 9: Baseline Model Structure.

To establish a strong and consistent benchmark for fraud detection, a two-layer baseline ensemble model was designed and applied across the group. This architecture shown in Figure 9 allows the performance of different aggregation strategies to be evaluated while keeping the underlying predictive models fixed.

Layer 1: Base Models (Common Across All Group Members)
The first layer of the ensemble uses several different models that are all trained on the same prepared dataset, rather than relying on just one model. This helps capture different types of fraudulent behaviour, as fraud can appear in many ways across transaction data. Before training, the data was cleaned and prepared so the models could work properly: categorical information was converted into numbers using one-hot encoding, numerical values were scaled to a similar range, and SMOTE was used to address the imbalance in the data, where fraudulent transactions occur far less frequently than legitimate ones. By balancing the dataset, it helps the models learn fraud patterns more effectively and reduces the risk of missing genuine fraud cases. Together, this approach helps the first layer detect a broader set of fraud patterns and produce more reliable results for the next stage of the ensemble

- **Logistic Regression**: was chosen as a clear and interpretable starting point. In the context of fraud detection, it provides understandable risk scores that show how individual features contribute to the likelihood of fraud, which is important for business transparency and regulatory compliance. The *max_iter = 1000* parameter ensures the model has sufficient iterations to converge on a stable solution, while *class_weight = "balanced"* compensates for the natural imbalance between fraudulent and legitimate transactions by giving more importance to fraud cases during training. This makes the model fairer and more reliable as a benchmark.

- **K-Nearest Neighbours (KNN)**: was included to capture similarity-based behaviour in transaction data. Fraudulent transactions often resemble each other in terms of amount, timing, or channel usage. The choice of *n_neighbors = 15* allows the model to make decisions based on a reasonably sized local neighbourhood, reducing sensitivity to noise, while *weights = "distance"* ensures that closer and more similar transactions have a stronger influence on predictions. This helps the model detect small clusters of suspicious behaviour that may not follow global trends.

- **Random Forest**: was selected to handle the complexity and non-linear nature of real-world fraud patterns, where risk often emerges from interactions between multiple features. Using *n_estimators = 400* improves prediction stability by averaging across many decision trees, while *max_depth = 12* and *min_samples_leaf = 20* control model complexity and reduce overfitting. The *class_weight = "balanced_subsample"* parameter further addresses class imbalance within each tree, making the model more sensitive to fraud without overreacting to noise.

- **Gaussian Naive Bayes**: was added to provide a fast and probabilistic view of fraud risk. Despite its simplifying assumptions, it performs efficiently on large datasets and often captures patterns missed by more complex models. Its lack of tuning parameters and low computational cost make it a valuable complementary model, increasing diversity within the ensemble and improving overall robustness.

Overall, these models were chosen because they complement one another rather than overlap. Together, they provide a balanced mix of interpretability, local pattern detection, non-linear modelling, and probabilistic reasoning. This diversity strengthens the ensemble and helps address the business challenge of detecting fraud accurately while minimising disruption to legitimate customers.

Layer 2: Aggregation Strategy (Individual Group Member Contribution)
The second layer of the ensemble focuses on how the predictions from the base models are combined to make a final fraud decision. While all group members use the same set of base models, each member applies a different aggregation approach at this stage. This allows us to directly study how different ways of combining model outputs affect fraud detection performance.

In practical fraud detection systems, decisions are often based on risk scores or probability estimates rather than simple class labels. Each aggregation function therefore works by combining the prediction scores or probabilities produced by the base models in Layer 1 and translating them into a final fraud likelihood or classification decision. This step is particularly important given the strong class imbalance in the data, where small changes in aggregation can significantly affect how many fraudulent transactions are detected or missed.

By keeping the base models fixed and changing only the aggregation method, the experimental setup isolates the impact of the aggregation strategy itself. This ensures that any differences in results are due to how predictions are combined, rather than differences in the underlying models. As a result, the approach maintains consistency across the group while allowing individual members to explore and justify alternative aggregation rules and provides a fair comparison.

Rationale for the Baseline Ensemble Design

The baseline ensemble was designed to act as a clear and reliable starting point for our fraud detection analysis. Its first role is to provide a strong reference model that allows us to fairly evaluate whether more advanced or alternative approaches actually improve performance. Rather than relying on a single classifier, the ensemble combines multiple models so that different types of fraud-related patterns can be captured. The second purpose of this design is to allow us to examine how different aggregation strategies influence fraud detection outcomes. In real-world fraud detection, the data is highly imbalanced and the cost of mistakes is unequal: missing a fraudulent transaction can lead to direct financial loss, while incorrectly blocking a legitimate customer can harm trust and revenue. Because of this, how model predictions are combined is just as important as the models themselves.

By keeping the base models fixed for all group members and varying only the aggregation method, we ensure a fair and structured comparison across experiments. This design also supports the group-based nature of the coursework, allowing everyone to work from a common modelling foundation while still contributing individually through different aggregation approaches.

Student 1: Lohit

3.1.1 Baseline Model with Entropy-Based Confidence Weighting Aggregation

In this aggregation approach [10] , each base model first produces a probability score for a transaction indicating how likely it is to be fraudulent. These probabilities are not treated as equally reliable. Instead, the method evaluates how confident each model is in its prediction by calculating entropy for every probability output.

Entropy is used as a measure of uncertainty in a model's prediction. When a model predicts a probability close to 0 or 1, it is expressing high confidence, and the entropy value is low. In contrast, when a probability is close to 0.5, the model is uncertain about whether the transaction is fraudulent or legitimate, resulting in high entropy. Entropy is calculated separately for each model and each transaction, allowing uncertainty to be assessed on a case-by-case basis rather than globally.

Once entropy values are obtained, they are converted into confidence weights by taking the inverse of entropy. This means that models with lower entropy (higher confidence) receive larger weights, while models with higher entropy (greater uncertainty) contribute less to the final decision. These weights are computed dynamically for every transaction, so a model's

influence can vary depending on how confident it is for a specific case.

The final fraud score is calculated as a weighted average of the individual model probabilities, where each probability is multiplied by its corresponding confidence weight and then normalised by the sum of all weights. As a result, confident predictions have a stronger impact on the final outcome, while uncertain predictions are automatically downweighted.

From a business perspective, this approach improves fraud detection by reducing missed fraud cases while avoiding unnecessary disruption to legitimate customers. It supports better risk management by prioritising confident signals and limiting the impact of uncertainty. Overall, entropy-based confidence weighting enhances decision quality and operational effectiveness, making it well suited for fraud detection in environments where fraud is rare but costly+

3.1.2 Model 2: Autoencoder with MLP Classifier

Alongside the baseline ensemble, an Autoencoder combined with a Multi-Layer Perceptron (MLP) classifier (inspired by this paper [5]) was implemented as a neural approach designed for transaction fraud data that is (i) high-dimensional after encoding, (ii) noisy, and (iii) strongly imbalanced. The core modelling strategy is to first learn a compact representation of "normal" transaction behaviour using an Autoencoder, and then use this compressed representation as the input to a supervised classifier that predicts fraud risk.[4]

This two-stage design is motivated by the operational reality of fraud detection: weak fraud signals are often spread across many variables, and the data pipeline (especially one-hot encoding) can create a sparse feature space that makes direct classification harder. By learning a lower-dimensional representation first, the downstream classifier can focus on the most informative patterns rather than the full noisy feature set.

Data Pre-processing Strategy

Before training, additional preprocessing was applied to ensure stable neural optimisation. Numerical features were standardised using z-score normalisation (zero mean, unit variance) so that variables with larger scales do not dominate gradient updates. Categorical variables were transformed using one-hot encoding, converting each category into a binary indicator. This avoids introducing false ordinal structure and enables categorical information to be used directly by the neural models. The encoding pipeline was also configured to safely handle unseen categories at evaluation time.

However, one-hot encoding substantially increases the number of input features, resulting in a high-dimensional and sparse representation. To reduce redundancy and suppress noise introduced at this stage, an Autoencoder was applied to learn a lower-dimensional latent representation that preserves the most informative structure in the data.

Autoencoder Architecture and Training

The Autoencoder used a symmetric encoder–decoder architecture with gradual compression. Given an input vector of dimension d, the encoder compresses the data in two stages, $d \to d/2 \to d/4$, rather than applying a single aggressive reduction. This stepwise approach is appropriate for fraud detection because subtle behavioural signals may be lost if dimensionality is reduced too abruptly.

ReLU activation functions were used in all hidden layers to introduce non-linearity and support stable optimisation. The decoder mirrors the encoder to reconstruct the original input, ensuring the latent representation retains sufficient information. A linear activation was used in the output layer to allow accurate reconstruction of continuous-valued features without artificial bounding.

Training minimised mean squared error (MSE) reconstruction loss using the Adam optimiser. The model was trained for up to 100 epochs with a batch size of 32, using a 20% validation split to monitor generalisation. Early stopping (patience = 10 epochs) based on validation loss was applied to prevent overfitting and to avoid learning noise rather than meaningful structure.

After training, only the **encoder** was retained. The encoder was then used to transform both the training and test datasets into compact latent feature representations, which were subsequently used as inputs to the fraud classifier.

MLP Classifier Design and Training

The MLP classifier was trained on the latent representations produced by the encoder rather than the original high-dimensional inputs. This improves training stability and encourages the classifier to focus on the most informative behavioural patterns captured during representation learning.

The MLP architecture consisted of two fully connected hidden layers with 64 and 32 neurons, respectively. This provides sufficient capacity to learn non-linear decision boundaries while controlling complexity to reduce overfitting risk. A sigmoid activation in the output layer produced a fraud probability score, which supports risk-based decision-making and flexible threshold selection.

The model was trained using binary cross-entropy loss with the Adam optimiser. To address severe class imbalance, class weights were applied so that fraudulent transactions contribute more to the loss function. This reduces bias toward the majority (legitimate) class and increases sensitivity to rare fraud cases, which is critical from a business-loss perspective. Training was run for up to 100 epochs with batch size 32, a 20% validation split, and early stopping (patience = 10) based on validation loss.

Tuning Threshold Strategy

Instead of using the default threshold of 0.5, a recall-driven thresholding strategy was applied. In fraud detection, the business cost of missed fraud typically exceeds the cost of investigating some false alarms. Therefore, the decision threshold was tuned to achieve a target recall of 0.8.

This was done by analysing the precision–recall trade-off and selecting the threshold that meets the recall requirement. Operationally, this aligns model decisions with risk tolerance and investigation capacity: higher recall reduces missed fraud at the expense of increased review workload.

Suitability for the Business Problem

This modelling strategy directly targets the practical difficulties of fraud detection: it manages high-dimensional inputs created by categorical encoding, reduces noise through learned latent representations, and captures complex non-linear behaviour patterns. Class weighting and recall-based threshold tuning further align the model with business objectives by prioritising fraud capture over headline accuracy.

Overall, the Autoencoder–MLP approach provides a robust alternative to traditional models, supporting better detection of subtle and rare fraud behaviour while maintaining control over operational impact through threshold-based decision policies.

<u>Student 2: Shivam</u>

3.1.3 Baseline Model with Max Probability (Recall-Oriented) Aggregation

In our baseline ensemble, each of the four Layer-1 models (LR, KNN, RF, GNB) outputs a fraud probability score for every transaction. Layer-2 then combines these model scores into a

single decision signal. For this baseline variant, we used Max Probability aggregation, which is designed to be recall-oriented.

Max Probability aggregation works. Layer 2 of the baseline ensemble combines the probability scores produced by the four base learners (Logistic Regression, KNN, Random Forest, and Gaussian Naïve Bayes) into one final fraud score. In the *Max Probability* aggregation variant, we define the final fraud score for a transaction x as:

$$s(x) = \max\{p_{\mathrm{LR}}(x),\, p_{\mathrm{KNN}}(x),\, p_{\mathrm{RF}}(x),\, p_{\mathrm{GNB}}(x)\}.$$

A binary decision is then made using a global decision threshold t (default $t = 0.5$ in our evaluation):

$$\hat{y}(x) = \begin{cases} 1 & \text{if } s(x) \geq t \\ 0 & \text{otherwise.} \end{cases}$$

Why this is recall-oriented. Max aggregation acts like a high-sensitivity "OR" rule across models: if *any* base model assigns a high fraud probability, the ensemble escalates the case. This design reduces missed fraud (false negatives) but usually increases false alarms (false positives), reflecting the well-known trade-off between precision and recall when the thresholding behaviour is made more aggressive.

Observed impact on our test set. Our dataset is severely imbalanced (fraud rate $\approx 2.21\%$: 1,322 fraud cases out of 59,939 total test instances). Under Max Probability aggregation, the fraud-class metrics were :

- Precision (fraud): 0.1006
- Recall (fraud): 0.9138
- F1-score (fraud): 0.1813
- ROC-AUC: 0.9373
- Average Precision (PR-AUC): 0.2880

The confusion matrix further explains the operational trade-off: the model correctly detected 1,208 fraud cases but flagged 10,795 legitimate transactions as fraud. In other words, Max Probability aggregation is highly effective at *capturing* fraud but generates a large investigation workload, so it is better suited to early-stage screening / risk triage rather than final automated blocking.

3.1.4 Model 2: Ripper

RIPPER (Repeated Incremental Pruning to Produce Error Reduction) is a rule-based classification algorithm that learns an ordered set of human-readable rules of the form:

IF (conditions) THEN fraud, else continue; finally default to non-fraud.

This makes RIPPER particularly attractive in fraud detection settings where decision transparency, auditability, and operational governance are important.

How RIPPER learns rules (high-level). RIPPER follows a "separate-and-conquer" (sequential covering) strategy:

1. **Grow a rule:** Start with an empty rule and greedily add conditions that improve separation between fraud and non-fraud, continuing until the rule stops covering negative examples (or further growth is not beneficial).
2. **Prune the rule (reduced-error pruning):** The grown rule is simplified using a pruning set to remove overfitting. A common pruning criterion maximises a score of the form $v = \frac{p-n}{p+n}$ where p and n represent (respectively) the number of positive and negative validation examples covered by the rule.
3. **Add rule to ruleset and remove covered positives:** Once a rule is accepted, the positive examples it covers are removed, and the algorithm repeats to learn additional rules until most positives are covered.
4. **Optimisation step:** RIPPER further improves the ruleset using an MDL-style heuristic by revisiting rules and simplifying them where possible, balancing ruleset complexity with error reduction.

Business relevance. The main advantage of RIPPER is interpretability: analysts can inspect the rules that trigger fraud flags, which supports explaining outcomes to stakeholders and building policy-aligned controls. However, rule-based learners can underperform on complex, highly non-linear fraud patterns because the decision boundary may not be easily expressed as a compact set of rules.

Performance on our test set. RIPPER produced the following fraud-class metrics:

- Precision (fraud): 0.2143
- Recall (fraud): 0.4992
- F1-score (fraud): 0.2999
- ROC-AUC: 0.7290
- Average Precision (PR-AUC): 0.1180

In confusion-matrix terms, RIPPER correctly detected 660 fraud cases but missed 662 fraud cases, while creating 2,420 false alarms. This confirms that RIPPER is more conservative than recall-first baselines (e.g., Max Probability), but its overall ranking/separation ability is substantially weaker than the stronger ML models in our group (see comparisons in the next section).

Student 3: Upasana

3.1.5 Baseline Model with Threshold-Aware Aggregation

For our baseline ensemble, we combined four standard models: Logistic Regression (LR), k-Nearest Neighbours (KNN), Random Forest (RF), and Gaussian Naïve Bayes (GNB). Each model gives a score between 0 and 1 for how likely a transaction is to be fraud. Instead of trusting only one model, we combine their scores to get a final decision. Our dataset is highly imbalanced (fraud cases are rare), and the main goal is *high recall*, meaning we would rather catch more fraud even if that creates more false alarms. In this situation, a simple average can be too cautious. For example, if one model is strongly confident that a case is fraud but the others are unsure, the average score can drop a lot, and the fraud case may be missed.

Idea of threshold-aware aggregation: Threshold-aware aggregation tries to solve this by treating each model fairly based on how it behaves. Different models produce different score patterns. Some models tend to give higher fraud scores, while others are more conservative. So instead of using the same cut-off for all models, we give each model its own *threshold* (its own "minimum score" for calling something suspicious).

How the thresholds are chosen: We do not choose thresholds using the test set. We split a small part of the training data as a validation set and use it to choose a threshold for each model. Since we care more about recall, we choose thresholds that help the model catch more fraud cases on the validation set (while still being reasonable).

How the final ensemble score is produced: When predicting on new data, we look at each model's score:

- If a model's score is *above its own threshold*, we treat it as a strong warning signal and give it more influence in the final combined score.
- If a model's score is *below its threshold*, we treat it as weak evidence and reduce its influence.

If none of the models pass their threshold for a specific transaction, we fall back to using the highest score among the models.

Why this is suitable for fraud detection: This method matches the fraud detection goal because it is designed to reduce missed fraud cases. It lets the ensemble react strongly when one or more models are confident, while also avoiding the situation where low-confidence scores from other models "drag down" the final result. In our experiments, this approach gave us a clearer way to control the precision-recall trade-off, which is important when deciding how strict the fraud detection system should be.

3.1.6 Model 2: Support Vector Machine (SVM)

For this experiment, I used a Support Vector Machine (SVM) classifier. The basic idea behind SVM is to separate the two classes using a straight decision boundary (a hyperplane) and to place that boundary so the gap (margin) between the classes is as wide as possible. The points that end up being most important are the ones closest to the boundary (called *support vectors*), because they are the ones that basically decide where the line goes. A larger margin generally helps the model generalise better, instead of fitting too closely to the training data.

Why I used a Linear SVM: The training dataset is quite large, and once the categorical columns are one-hot encoded, the number of features grows, and the data becomes sparse. Linear SVMs handle this situation well and train quickly. When I tried an RBF SVM, it took too long to run on the CPU, so it was not really workable for this coursework setup. Using a linear SVM gave me a model that I could train and iterate on without waiting hours.

Pre-processing: Pre-processing mattered a lot because SVMs are sensitive to the scale of the inputs.The categorical columns (`country`, `bin_country`, `merchant_category`) were converted using *one-hot encoding.*The numeric features were standardised (mean 0, standard deviation 1). Without this, features with larger ranges can dominate the optimisation and affect the margin.

Class imbalance handling: This is a *fraud* dataset, so the classes are imbalanced (fraud is the minority). I used *class weighting* in the SVM. In simple terms, mistakes in the fraud class were given a higher weight than mistakes in the non-fraud class, so the model is pushed to pay more attention to fraud cases. I did not use SMOTE because we one-hot encode categorical variables, the feature space becomes sparse, and creating synthetic samples in that space can produce unrealistic combinations.

Calibration and choosing a threshold: A linear SVM does not naturally give probabilities; it mainly gives a decision score (how far a point is from the separating hyperplane). Since I wanted to plot ROC/PR curves and also adjust how strict the classifier is, I calibrated the SVM scores using sigmoid calibration (Platt scaling) with *CalibratedClassifierCV*. This gives probability-like outputs that I can threshold. I split the training data into a training and validation part, and used the validation set to choose a threshold from the precision-recall curve.

Overall, the linear SVM worked well as a scalable model for this dataset, and the combination of class weighting and threshold tuning helped improve fraud recall compared to using the default settings.

Student 4: Amarjeet

3.1.7 Baseline Model with Weighted Average Aggregation

In this method [7], each base model first produces a probability score representing how likely a transaction is to be fraudulent. These probability scores are then multiplied by predefined weights that represent the relative importance of each model. Models that are known to capture complex fraud patterns more effectively are given higher weights, while models that tend to perform less strongly are given lower weights. The final fraud score is calculated as the weighted sum of these probability values.

The weighting scheme reflects practical experience in fraud detection. Models such as Random Forest and Logistic Regression are typically strong performers on transactional data: Random Forest is effective at capturing non-linear interactions between features, while Logistic Regression provides stable and well-calibrated probability estimates. As a result, these models contribute more heavily to the final decision. Other models, such as K-Nearest Neighbours and Gaussian Naive Bayes, still provide valuable complementary signals but are assigned lower weights to limit their influence when they are less reliable.

From a business perspective, weighted average aggregation offers a stable and predictable decision-making process. Because the weights are fixed, the ensemble behaves consistently across transactions, which is important in operational fraud systems where decisions must be explainable and repeatable. This approach reduces the risk that weaker model predictions dominate the final outcome, while still benefiting from the diversity of multiple models.

Overall, weighted average aggregation provides a balanced compromise between simplicity and performance. It leverages the strengths of high-performing models, incorporates additional signals from weaker models, and produces a final fraud score that is robust, interpretable, and well suited to the business goal of detecting fraud while minimising unnecessary disruption to legitimate customers.

3.1.8 Model 2: XGBoost (Extreme Gradient Boosting))

XGBoost (Extreme Gradient Boosting) was implemented as an advanced supervised learning model to address the complex and highly imbalanced nature of the fraud detection problem[2]. XGBoost is an efficient implementation of gradient-boosted decision trees and is particularly

well suited to large-scale, tabular transactional data where fraud patterns are driven by non-linear interactions between multiple features. This model was selected as it provides strong predictive performance while offering built-in mechanisms to control overfitting, handle class imbalance, and scale efficiently to large datasets, all of which are essential requirements in real-world fraud detection systems.

Rationale for Selecting XGBoost: it was chosen primarily because of its ability to capture complex, non-linear relationships that commonly arise in fraudulent behaviour. Fraud rarely depends on a single variable; instead, it emerges from interactions between features such as transaction amount, timing, channel, and behavioural indicators. Decision tree–based boosting models are particularly effective at learning such interactions. In addition, XGBoost includes native support for handling class imbalance through weighting mechanisms, allowing the model to prioritise fraudulent transactions during training. The algorithm also incorporates regularisation directly into the learning objective, helping prevent overfitting in high-dimensional financial datasets. Finally, XGBoost is optimised for speed and memory efficiency, making it suitable for large transaction volumes and practical deployment scenarios.

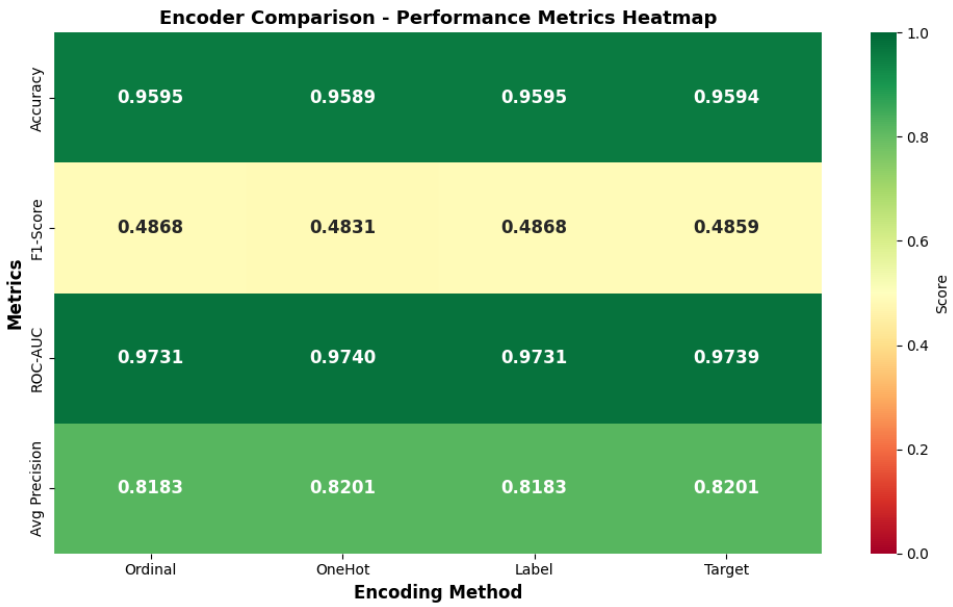


Figure 10: Different Encoding comparison

Categorical Encoding Strategy: The transaction dataset used for modelling was generally preprocessed prior to this stage; however, additional preprocessing steps specific to XGBoost were applied in Python. Since XGBoost requires numerical input, categorical variables were encoded using several alternative strategies, including ordinal encoding, label encoding, target encoding, and one-hot encoding. These encoding approaches were evaluated systematically to assess their impact on fraud detection performance. All encodings were applied consistently to the training and test datasets to prevent information leakage. Based on empirical results as shown in Figure 10, one-hot encoding was selected for the final model as it produced the strongest discriminative performance, achieving the highest ROC-AUC and average precision scores among the tested approaches. This indicates that preserving categorical distinctions without imposing artificial ordinal relationships was most effective for this dataset.

Handling Class Imbalance A major challenge in this fraud detection task is the extreme imbalance between legitimate and fraudulent transactions. Rather than applying resampling

techniques such as SMOTE, which can distort the original data distribution and introduce unrealistic synthetic samples, class imbalance was handled directly within the XGBoost framework. This was achieved by applying a class weighting strategy that increases the penalty for misclassifying fraudulent transactions. The weighting factor was calculated as the ratio of non-fraudulent to fraudulent transactions and incorporated into the model's learning objective. This encourages the model to focus more strongly on the minority fraud class while still learning from the full dataset.

Model Configuration and Training Strategy The XGBoost model was configured with hyperparameters chosen to balance learning capacity and generalisation. The maximum tree depth was set to six, allowing the model to capture meaningful feature interactions without producing overly complex trees. A learning rate of 0.05 was used in combination with 300 boosting iterations, enabling the model to learn gradually and converge more reliably. To reduce overfitting, stochastic boosting was applied by subsampling both training instances and features at a rate of 0.8 for each tree. These design choices help ensure stable performance on unseen transaction data.

For evaluation, standard accuracy was not prioritised, as it can be misleading in highly imbalanced classification problems. Instead, the Area Under the Precision–Recall Curve (AUCPR) was used as the primary evaluation metric. AUCPR provides a more informative measure of performance by focusing on the trade-off between precision and recall for the minority fraud class, which aligns closely with the business objective of maximising fraud detection while controlling false alarms.

The XGBoost model outputs probability scores rather than direct class labels, allowing flexible threshold-based decision making. This is particularly important in fraud detection systems, where decision thresholds can be adjusted depending on operational constraints and risk tolerance. In addition, feature importance analysis was conducted to identify which variables most strongly influence fraud predictions. This provides valuable insights for business stakeholders, such as risk analysts and fraud teams, by highlighting key drivers of fraudulent behaviour.

Student 5: Nneamaka

3.1.9 Baseline Model with Trimmed Mean aggregation

The baseline is an ensemble model which was developed using Trimmed Mean Aggregation, and consists of base models including logistic regression, KNN, random forest, and gaussian naïve bayes. This approach was taken in order to combine the strengths of multiple classifiers as individual models have the tendency to produce unstable or extreme predictions, especially in fraud datasets that are highly imbalanced. Combining multiple models is beneficial to fraud detection systems because they tend to be more robust. Meta-models were not trained, but the trimmed mean ensemble works at probability level by aggregating output from each of the base models while remaining simple and computationally efficient.
For each transaction, the aggregation method works in the following manner:
i. The probabilities of fraud were obtained from the multiple base models.
ii. The highest and lowest probabilities were removed.
iii. The average of the remaining probabilities were taken to produce a final fraud score.

The trimmed mean aggregation approach reduces the amount of extreme and noisy predictions while maintaining probabilistic decision-making and improving robustness.
For preprocessing, numeric features were scaled and the categorical variables (country, BIN country, and merchant category) were one hot encoded. This was implemented in python on only the training data to prevent information leakage. Dimensionality reduction was not applied in order to allow the dataset learn from the full feature set.
The implementation of the trimmed mean aggregation ensemble involved gathering probability

outputs from the base models and applying trimmed mean aggregation at the time of prediction. The final ensemble probability score was then converted to a binary fraud prediction using a fixed decision threshold.
Each base model within the ensemble was trained independently using the same training dataset. After training, fraud probability outputs from each of the base models were gathered for ever transaction in the test set. The trimmed mean aggregation was then applied to each sample at the time of prediction. Like the LDA model, the decision threshold was set to 0.5 for the aggregated probabilities to generate the final fraud predictions. Keeping the threshold consistent allows for fair comparison across models and avoids introducing bias through optimisation.

3.1.10 Model 2: Linear Discriminant Analysis (LDA)

The Linear Discriminant Analysis model was chosen due to its simplicity and interpretability as a baseline classifier. It is suitable for binary classification problems with structured numeric data. It is a linear and generative model that discovers the optimal linear combination of features for class separation [9], in this case, fraudulent and non-fraudulent transactions. In this business context, LDA proves to be useful to stakeholders such as risk analysts and compliance teams because its coefficients can be interpreted clearly and directly thereby providing insight into how different variables influence the risk of fraud.[6]

The transactions dataset used for modelling was preprocessed generally before modelling using the R language, but some more preprocessing specific to the LDA model including scaling numeric features and encoding the categorical variables (country, BIN country, and merchant category), were implemented in python on only the training data to prevent information leakage. SMOTE was not applied because LDA models depend on assumptions about sample independence and distribution, therefore, including synthetic observations could produce wrong probability estimates [1]. Instead, the class balance was handled using the appropriate evaluation metrics. This was to ensure compatibility with the LDA model. Dimensionality reduction was not applied in order to allow the dataset learn from the full feature set. The LDA model assumes that each class follows a multivariate normal distribution and that classes share a common covariance matrix. This might not be the exact case for fraud transactional data, but it is known to perform very well when features are standardised and engineered properly.

Before modelling, the dataset was split into training and test sets. The LDA model was implemented in Python using scikit-learn and fitted on the training dataset using all the numerical and encoded categorical features. It learnt a linear combination of features that separate fraudulent and non-fraudulent transactions. Its outputs are not class predictions, but probabilities, which are important for evaluation in imbalanced classification problems such as this. The decision threshold was set to 0.5 to convert the predicted probabilities into class labels. The threshold shows the neutral trade-off between precision and recall and provides a clear baseline which can be compared with other models.

A major advantage of the LDA is its interpretability. Its model coefficients were analysed to find out which features positively or negatively influence the likelihood of frauds occurring. This is very important because it provides insight into why a transaction is flagged as fraudulent.

4 Performance Evaluation

Baseline Model Comparison

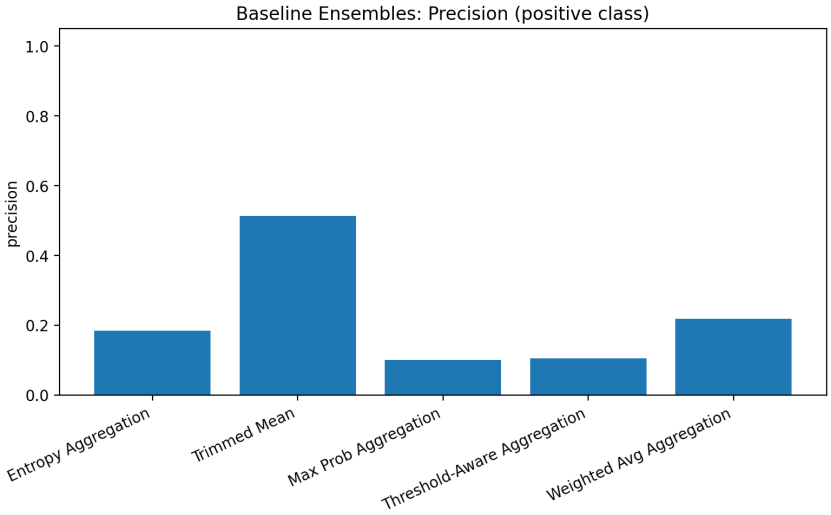


Figure 11: Baseline Model Precision.

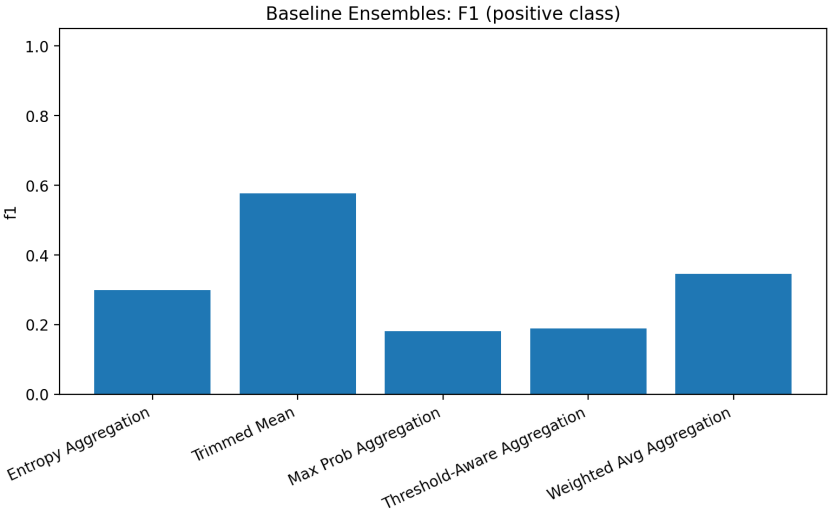


Figure 12: Baseline Model F1.

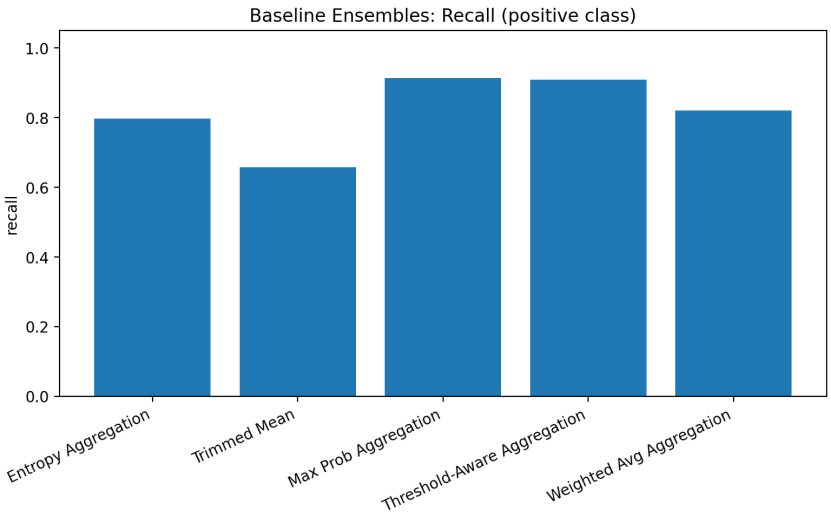


Figure 13: Baseline Model Recall.

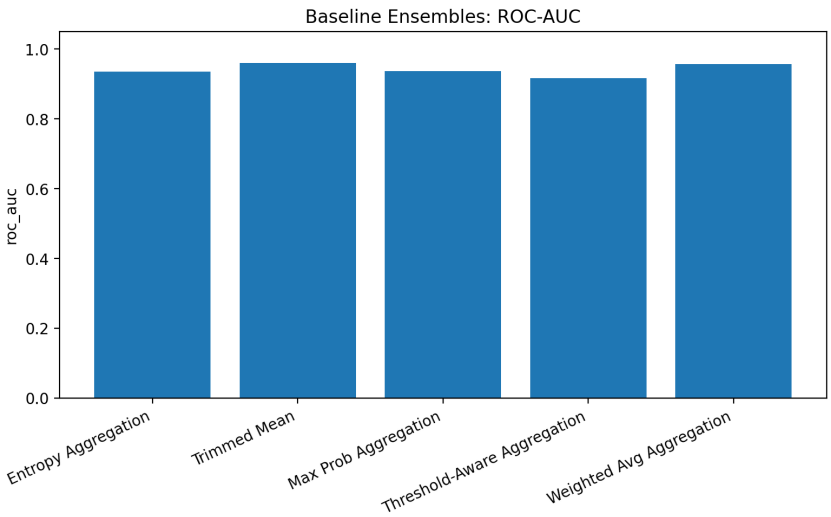


Figure 14: Baseline Model ROC-AUC Score.

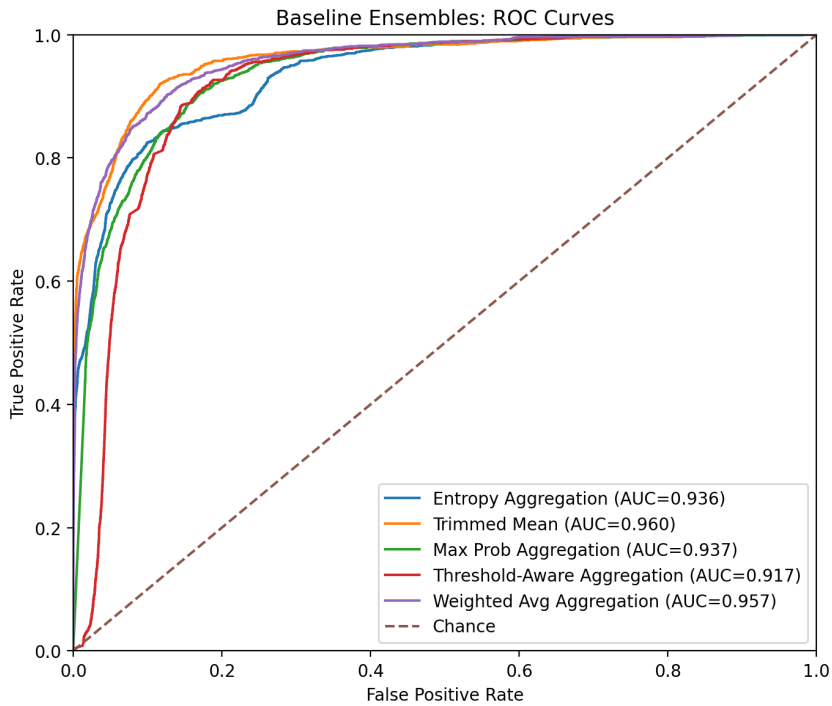


Figure 15: Baseline Model ROC Curves.

Best Baseline Model vs Advance Model

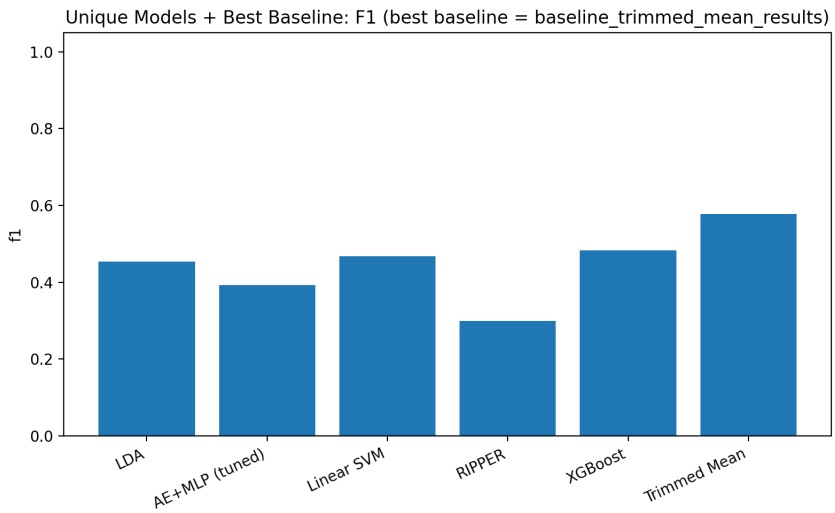


Figure 16: F1 Score Comparison: Unique Model vs Best Baseline.

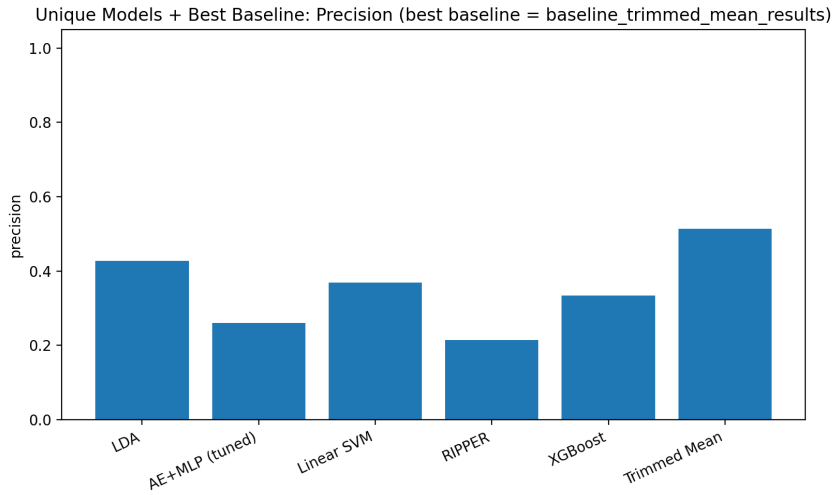


Figure 17: Precision Comparison: Unique Model vs Best Baseline.

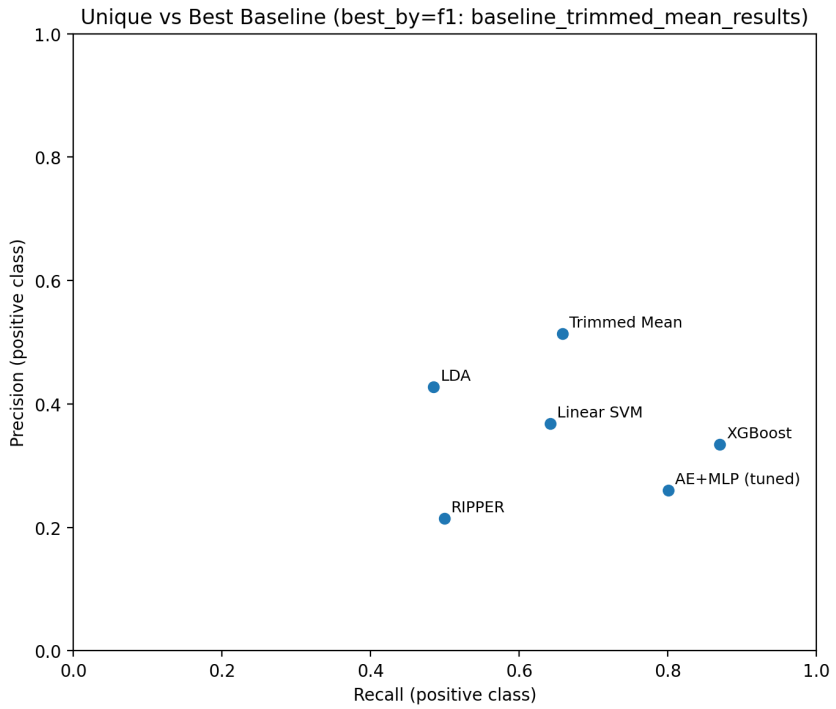


Figure 18: Precision–Recall Comparison: Unique Model vs Best Baseline.

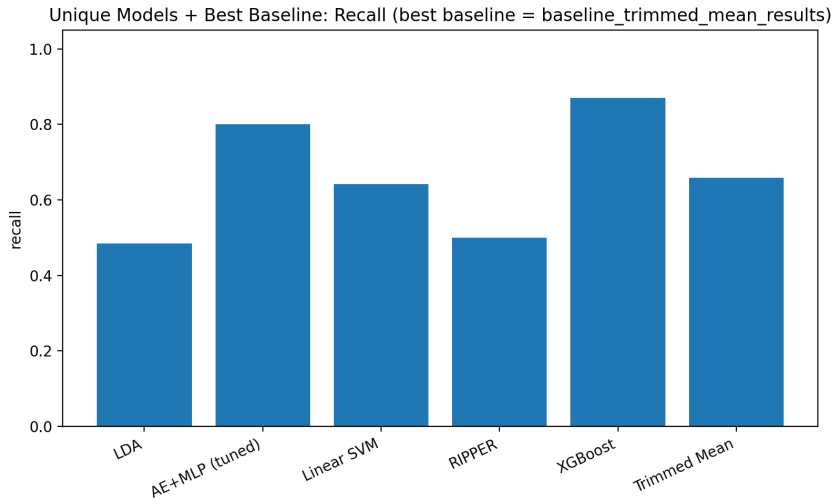


Figure 19: Recall Comparison: Unique Model vs Best Baseline.

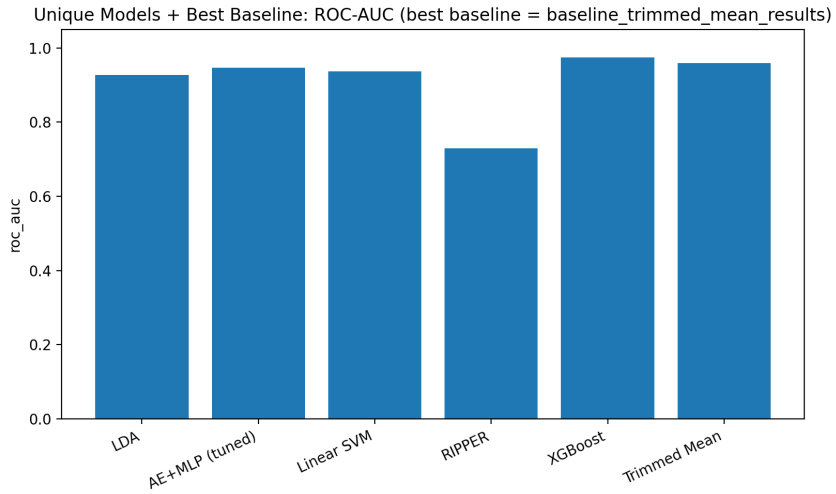


Figure 20: ROC-AUC Comparison: Unique Model vs Best Baseline.

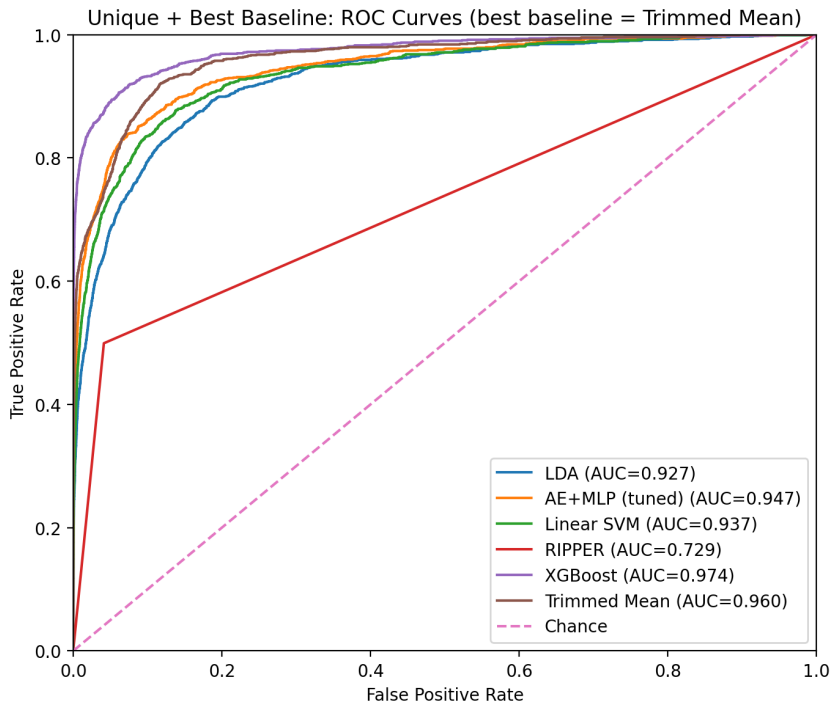


Figure 21: ROC Curve Comparison: Unique Model vs Best Baseline.

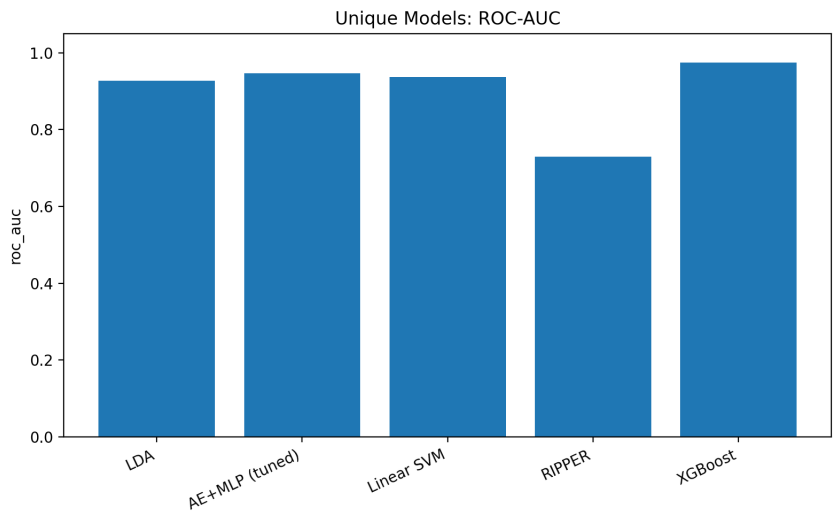


Figure 22: ROC-AUC comparison across unique models.

Student 1: Lohit

4.0.1 Evaluation Step for Both Models

1. Both models output fraud probability scores, with the baseline ensemble using entropy-based weighted aggregation and the Autoencoder-based model using an MLP classifier on encoded features.

2. Probabilities are converted into binary predictions using a decision threshold, with 0.5 used as the default for initial evaluation.

3. Precision, recall, and F1-score are computed to measure fraud detection effectiveness and false alert rates.

4. ROC–AUC and Average Precision (PR–AUC) are calculated to assess threshold-independent performance under class imbalance.
5. A confusion matrix is generated to quantify false positives and false negatives and assess business impact.
6. ROC and Precision–Recall curves are produced to analyse model behaviour across different thresholds.
7. For the Autoencoder-based model, the decision threshold is tuned to achieve a target recall of 0.8, and performance is re-evaluated.
8. All evaluation outputs are saved in JSON format to support reproducibility and comparison.

4.0.2 Individual Assessment of the Baseline Model with Entropy-Based Aggregation

The baseline ensemble model achieves an overall accuracy of 91.7% and a ROC–AUC of 0.94, indicating strong overall discrimination between fraudulent and non-fraudulent transactions.

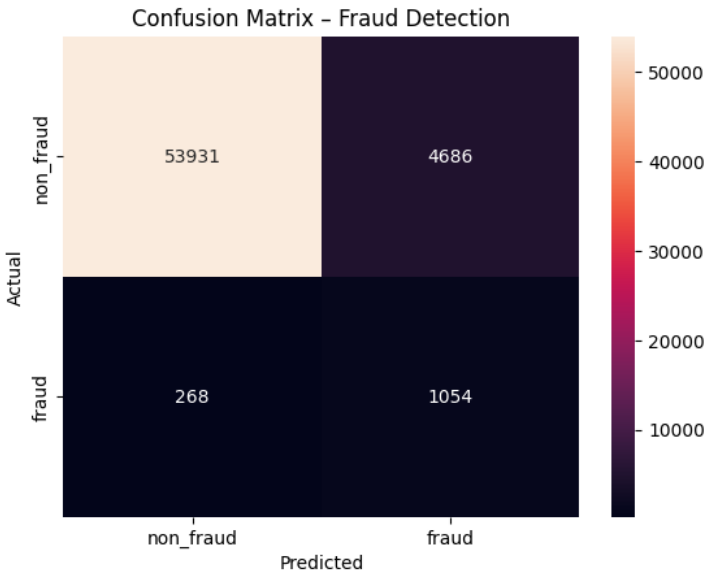


Figure 23: Baseline Model with Entropy Aggregation Confusion Matrix

The Figure 23 confusion matrix shows that 53,931 non-fraud transactions are correctly classified, while 4,686 legitimate transactions are incorrectly flagged as fraud. This level of false positives may increase operational review effort and customer friction.

For fraudulent transactions, the model correctly identifies 1,054 cases and misses 268 cases. This corresponds to a high fraud recall of 0.80, demonstrating that the model is effective at detecting most fraud events, which is important for reducing financial losses.

However, fraud precision is low at 0.18, meaning that a large proportion of transactions flagged as fraud are actually legitimate. This indicates that the model is conservative and prioritises fraud detection over alert accuracy.

The Average Precision score of 0.53 reflects the difficulty of fraud detection under severe class imbalance. Overall, the baseline model is suitable as an initial risk-screening mechanism but would require additional verification steps before automated decision-making.

4.0.3 Comparison of Baseline Model with Entropy-Based Aggregation function with Other Baseline Models

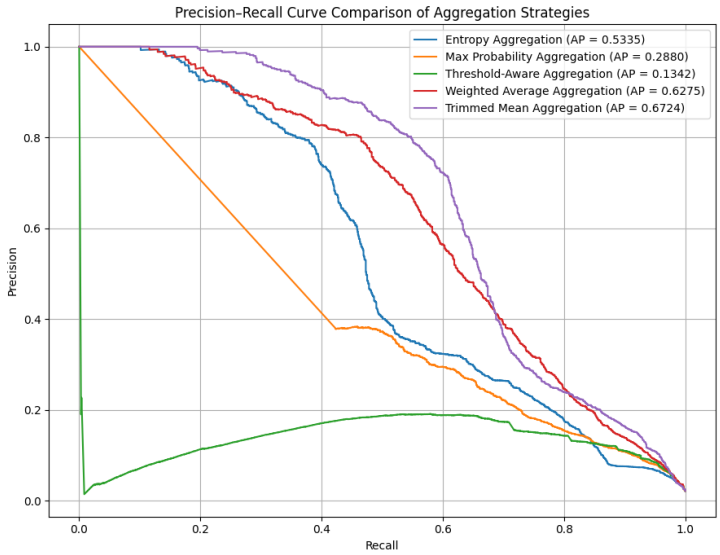


Figure 24: Baseline Model Precision Recall

Figure 24 shows that the baseline model with entropy-based aggregation achieves an Average Precision of 0.53. This is substantially better than threshold-aware aggregation (AP = 0.13) and max-probability aggregation (AP = 0.29), indicating that entropy weighting provides a more reliable balance between fraud detection and false alerts. However, it is outperformed by trimmed mean aggregation (AP = 0.67) and weighted average aggregation (AP = 0.63), which offer stronger overall precision–recall trade-offs.

From a practical standpoint, this means that the entropy-based model produces more trustworthy fraud alerts than simpler aggregation strategies, but still generates more false positives than the strongest baseline, trimmed mean aggregation.

Figure 13 highlights differences in fraud recall. The entropy-based aggregation achieves a recall of approximately 0.80, meaning that four out of five fraudulent transactions are successfully detected. This is lower than threshold-aware aggregation, which achieves a very high recall of around 0.91, but higher than trimmed mean aggregation, which detects approximately 66% of fraud cases. For a business, this places the entropy-based model in a middle ground: it captures most fraud while avoiding the extreme false-alert rates associated with threshold-aware aggregation.

Figure 15 shows that the entropy-based model achieves a strong ROC–AUC of approximately 0.94. This indicates good overall discrimination between fraudulent and legitimate transactions and confirms that the model can effectively rank transactions by risk. While trimmed mean aggregation achieves a slightly higher ROC–AUC (approximately 0.96), the entropy-based model performs noticeably better than threshold-aware aggregation (ROC–AUC $\approx$ 0.92).

Overall, the entropy-based aggregation model represents a balanced baseline approach. Compared to threshold-aware aggregation, it significantly improves alert quality while still maintaining high fraud detection capability. Compared to trimmed mean aggregation, it offers higher fraud coverage at the cost of lower precision. From a business perspective, this makes the entropy-based model suitable for risk scoring or prioritisation stages, where both fraud capture and alert quality are important, while trimmed mean aggregation is better suited for final automated decision-making.

4.0.4 Individual Assessment of AutoEncoder with Multi-Layer Preceptron

Table 1: Class-wise Performance at Different Decision Thresholds

Class	Threshold	Recall	F1-Score	Precision	ROC-AUC	PR-AUC
0 (Non-Fraud)	0.50	0.9248	0.9591	0.9961	0.9471	0.6054
	0.645	0.9487	0.9714	0.9953	0.9471	0.6054
1 (Fraud)	0.50	0.8396	0.3246	0.2012	0.9471	0.6054
	0.645	0.8003	0.3928	0.2603	0.9471	0.6054

Table 1 summarises the class-wise performance of the Autoencoder with MLP before and after threshold tuning. The comparison highlights how threshold selection directly influences fraud detection effectiveness and operational impact.

At the default threshold of 0.5, the model achieves a fraud recall of 0.84, meaning that approximately 84% of fraudulent transactions are correctly detected. This high recall reduces potential fraud losses but is accompanied by relatively low fraud precision of 0.20, indicating a high number of false fraud alerts. From a business perspective, this configuration prioritises fraud capture over alert accuracy, increasing operational review costs and customer friction.

After threshold tuning to 0.645, fraud recall decreases slightly to 0.80, meaning that around 80% of fraud cases are still detected. However, fraud precision improves from 0.20 to 0.26, indicating that a higher proportion of flagged transactions are genuinely fraudulent. This represents a meaningful reduction in false positives while maintaining strong fraud detection capability.

For non-fraud transactions, recall improves from 0.92 to 0.95 after threshold tuning, showing that more legitimate transactions are correctly classified. This directly supports a smoother customer experience by reducing unnecessary transaction declines.

The ROC–AUC and PR–AUC remain constant at 0.95 and 0.61 respectively across both thresholds, indicating that the model's underlying ability to rank risky transactions does not change. Instead, threshold tuning adjusts how conservatively or aggressively the model is applied in practice.

Overall, threshold tuning shifts the Autoencoder-based model from a highly sensitive fraud detection mode to a more balanced operational configuration. While the default threshold is suitable for aggressive fraud screening, the tuned threshold offers a better trade-off between fraud prevention and customer impact, making it more appropriate for real-world deployment.

4.0.5 Comparasion between Threshold Tuned AutoEncoder with MLP and other Advance Models including Best Basline Model

Figure 16 shows that the threshold-tuned Autoencoder with MLP achieves a positive-class F1-score of approximately 0.39. This outperforms RIPPER (F1 $\approx$ 0.30) and is comparable to Linear SVM (F1 $\approx$ 0.47), but remains below the best baseline, Trimmed Mean aggregation, which achieves the highest F1-score of approximately 0.58. This indicates that while the Autoencoder improves over some traditional models, the best baseline still provides the strongest overall balance between precision and recall.

From a precision perspective, Figure 17 shows that the Autoencoder with MLP achieves a precision of approximately 0.26. This is higher than RIPPER (precision $\approx$ 0.21) but lower than LDA (precision $\approx$ 0.43), Linear SVM (precision $\approx$ 0.37), and Trimmed Mean aggregation (precision $\approx$ 0.51). For a business user, this means that although the Autoencoder reduces false positives compared to weaker models, it still generates more false alerts than the best baseline.

Figure 19 highlights fraud recall, where the Autoencoder with MLP performs strongly with a recall of approximately 0.80. This is higher than LDA (recall $\approx$ 0.49), Linear SVM (recall $\approx$ 0.64), RIPPER (recall $\approx$ 0.50), and Trimmed Mean aggregation (recall $\approx$ 0.66), and is second only to XGBoost (recall $\approx$ 0.87). From a business perspective, this demonstrates that the

Autoencoder is particularly effective at capturing fraudulent transactions, reducing potential fraud losses.

Figure 20 shows that the Autoencoder with MLP achieves a ROC–AUC of approximately 0.95. This indicates strong overall discrimination and places it above LDA (ROC–AUC ≈ 0.93), Linear SVM (ROC–AUC ≈ 0.94), and RIPPER (ROC–AUC ≈ 0.73). While XGBoost achieves the highest ROC–AUC (approximately 0.97), the Autoencoder performs comparably to the best baseline, Trimmed Mean aggregation (ROC–AUC ≈ 0.96).

The ROC curves in Figure 21 further confirm that the threshold-tuned Autoencoder consistently performs well above the chance line and shows stable behaviour across false-positive rates, though XGBoost and Trimmed Mean aggregation achieve slightly stronger separation.

Overall, the threshold-tuned Autoencoder with MLP offers a strong compromise between fraud detection and alert quality. Compared to the best baseline (Trimmed Mean aggregation), it achieves higher fraud recall at the cost of lower precision. From a business standpoint, this makes the Autoencoder particularly suitable for risk prioritisation and analyst-driven review stages, while Trimmed Mean aggregation remains more appropriate for final automated decision-making where false positives must be minimised.

Student 2: Shivam

Comparison 1: My baseline (Max Probability) vs other baseline aggregation methods: My baseline ensemble used Max Probability aggregation, which is a recall-first strategy. On the fraud class it achieved:

$$\text{Precision} = 0.101, \quad \text{Recall} = 0.914, \quad \text{F1} = 0.181, \quad \text{ROC-AUC} = 0.937, \quad \text{PR-AUC} = 0.288.$$

From the confusion matrix, it detected 1,208/1,322 fraud cases (FN=114) but produced 10,795 false alarms (FP=10,795). This confirms the expected behaviour: very high fraud capture, but low alert quality.

When compared against the other four baseline aggregation methods, Figure 11, Figure 12, Figure 13, and Figure 15 show that Max Probability sits at the extreme recall-focused end:

- **Threshold-Aware aggregation:** Precision=0.106, Recall=0.909, F1=0.190, ROC–AUC=0.917, PR-AUC=0.134; FP=10,156; FN=120. This baseline is similarly recall-first but has even weaker PR-AUC than Max Probability, suggesting poorer precision-recall behaviour overall.
- **Entropy aggregation:** Precision=0.184, Recall=0.797, F1=0.298, ROC-AUC=0.936, PR-AUC=0.533; FP=4,686; FN=268. This provides a more stable middle-ground: it gives up some recall compared to Max Probability but more than doubles precision (0.184 vs 0.101) and improves PR-AUC substantially (0.533 vs 0.288).
- **Weighted Average aggregation:** Precision=0.218, Recall=0.820, F1=0.345, ROC-AUC=0.957, PR-AUC=0.628; FP=3,878; FN=238. This improves both alert quality and ranking metrics relative to Max Probability, while still maintaining strong recall.
- **Trimmed Mean aggregation (best baseline balance):** Precision=0.514, Recall=0.658, F1=0.577, ROC-AUC=0.960, PR-AUC=0.672; FP=823; FN=452. This is the strongest baseline in terms of precision and F1, producing far fewer false alarms than the recall-first baselines, but it misses more fraud cases due to its conservative behaviour.

Takeaway (baselines): If the business goal is *maximum fraud capture*, Max Probability performs best on recall (0.914), but it creates the highest operational burden (FP=10,795). If the goal is *balanced decision-making* with credible alerts, Trimmed Mean and Weighted Average are stronger baselines (PR-AUC 0.672 and 0.628 respectively), with dramatically lower false positives.

Comparison 2: My unique model (RIPPER) vs other unique supervised models: My unique model was RIPPER [3], a rule-based learner designed for interpretability (IF-THEN rules). Its fraud-class performance was:

$$\text{Precision} = 0.214, \quad \text{Recall} = 0.499, \quad \text{F1} = 0.300, \quad \text{ROC–AUC} = 0.729, \quad \text{PR–AUC} = 0.118,$$

with FP=2,420 and FN=662 (660 frauds detected, 662 missed). This shows RIPPER captures some stable fraud patterns, but misses roughly half of all fraud cases and has weak ranking quality (low PR-AUC).

Comparison against the other unique models in the group:

- **XGBoost (best unique overall):** Precision=0.334, Recall=0.870, F1=0.483, ROC-AUC=0.974, PR-AUC=0.820; FP=2,289; FN=172. XGBoost dominates RIPPER on both recall and ranking quality, catching far more fraud (0.870 vs 0.499) while maintaining comparable false-positive volume.
- **Threshold-tuned AE+MLP:** Precision=0.260, Recall=0.800, F1=0.393, ROC–AUC=0.947, PR–AUC=0.605; FP=3,007; FN=264. This offers high recall and good PR–AUC, indicating strong performance for analyst-driven review stages.
- **Linear SVM (threshold adjusted):** Precision=0.369, Recall=0.641, F1=0.468, ROC–AUC=0.937, PR–AUC=0.538; FP=1,453; FN=474. This provides a cleaner precision/recall balance than RIPPER and substantially stronger ranking behaviour.
- **LDA:** Precision=0.427, Recall=0.485, F1=0.454, ROC–AUC=0.927, PR–AUC=0.462; FP=859; FN=681. LDA has better precision than RIPPER, but recall is similar/slightly lower; overall ranking is still stronger than RIPPER.

Takeaway (unique models): RIPPER is the weakest unique model on ROC–AUC and PR–AUC (0.729 and 0.118) but remains valuable for explainability and governance (rules can be communicated and audited). In terms of detection quality and ranking performance on this dataset, XGBoost is clearly the strongest individual supervised model (PR–AUC 0.820).

Comparison 3: My baseline and my unique vs all other models (answering the research question):

Research question: *Which supervised learning approach (individual models vs stacking/ensemble approaches) yields the best fraud detection performance on an imbalanced e-commerce dataset?* To answer this, we compare the **best individual model(s)** against the **best ensemble baselines** using fraud-focused metrics. Because the data is imbalanced (fraud is rare), we give particular weight to PR-AUC and the precision/recall trade-off.

Best individual supervised approach (unique models): XGBoost (Figure 13) achieves the strongest overall ranking and fraud capture: PR-AUC=0.820, ROC-AUC=0.974, Recall=0.870, Precision=0.334, with FN=172 and FP=2,289. This indicates the best ability to prioritise truly fraudulent transactions in a ranked list.

Best ensemble baseline approach (aggregation/stacking-style combining): Among baselines, Trimmed Mean (Figure 17 and Figure 19) is the best balanced ensemble (Precision=0.514, Recall=0.658, F1=0.577, PR–AUC=0.672) with very low FP (823). However, it trades away recall compared to XGBoost (0.658 vs 0.870) and misses more fraud (FN=452 vs 172).

Where my two models fit in this picture:

- My Max Probability baseline is an extreme recall-first ensemble (Recall=0.914, FN=114) but has low precision (0.101) and moderate PR–AUC (0.288), creating the largest alert workload (FP=10,795).
- My RIPPER model is interpretable, but has weaker detection and ranking performance (Recall=0.499, PR–AUC=0.118).

Conclusion: On this dataset, advanced individual supervised models (especially XGBoost) provide the best overall fraud detection performance when performance is defined as ranking quality and strong recall under imbalance (highest PR-AUC and ROC-AUC). At the same time, ensemble aggregation baselines can deliver the best alert quality (highest precision and F1), as seen with Trimmed Mean. Therefore, the answer depends on the business definition of "best":

- If the priority is catching as much fraud as possible: recall-first ensemble approaches (e.g., Max Probability / Threshold-Aware) perform best.
- If the priority is balanced performance and credible alerts: the strongest individual model (XGBoost) is best overall on PR–AUC, while Trimmed Mean is best on precision/F1.

This supports a practical staged deployment: ensembles can be used for high-sensitivity screening, while strong individual learners provide decision-grade scoring; stacking/ensemble methods are specifically designed to combine multiple learners and can be advantageous depending on the objective.

Student 3: Upasana

This section explains how I evaluated the two models I worked on: (1) Baseline Ensemble with Threshold-Aware Aggregation and (2) Linear SVM. Because *fraud* is the minority class, I focused mainly on how well the models detect fraud (recall), but I also report precision, F1, ROC-AUC, and PR-AUC to show the trade-offs.

Evaluation steps: To keep comparisons fair, I followed the same steps for all models:
1. Same train/test split: all models were evaluated on the same held-out test set.
2. Probability outputs: models that can output probabilities were evaluated using their fraud probability scores.
3. Decision threshold: probabilities were converted to class labels using a threshold. For my Threshold-Aware baseline, I used a fixed threshold of 0.5. For Linear SVM, I used a lower threshold (0.1137) to make it more recall-focused.
4. Metrics reported: confusion matrix, fraud-class precision, recall, F1, ROC-AUC, and PR-AUC (Average Precision).
5. Group comparisons: I used ROC curves and bar charts (recall, precision, F1) to compare my models against the group's other baseline ensembles and unique models.

Results for Baseline Ensemble with Threshold-Aware Aggregation: This model is designed to be recall-first, meaning it tries to catch as many fraud cases as possible. On the test set, it achieved *fraud recall* = 0.909, which means it detected most fraud cases. The downside is that it produced many false alarms, with *fraud precision* = 0.106 and *fraud F1* = 0.190. In terms of ranking ability, it achieved *ROC-AUC* = 0.917 and *PR-AUC* = 0.134. The confusion matrix helps explain the precision/recall trade-off. Out of 1322 fraud cases, the model correctly detected 1202 and missed 120. However, it also flagged 10156 normal transactions as fraud. This is the main cost of pushing recall high in an imbalanced fraud dataset.

Results for Linear SVM: For Linear SVM, I chose a lower decision threshold (0.1137) to push recall higher. With this threshold, Linear SVM achieved *fraud recall* = 0.641 and *fraud precision* = 0.369, giving a higher *fraud F1* = 0.468 than the recall-first baseline. It also performed strongly on threshold-independent metrics with *ROC-AUC* = 0.937 and *PR-AUC* = 0.538.

Table 2: Summary of my two models on the fraud (positive) class.

Model	**Thr.**	**Recall**	**Prec.**	**F1**	**ROC-AUC**	**PR-AUC**
Threshold-Aware Baseline	0.50	0.909	0.106	0.190	0.917	0.134
Linear SVM (unique)	0.114	0.641	0.369	0.468	0.937	0.538

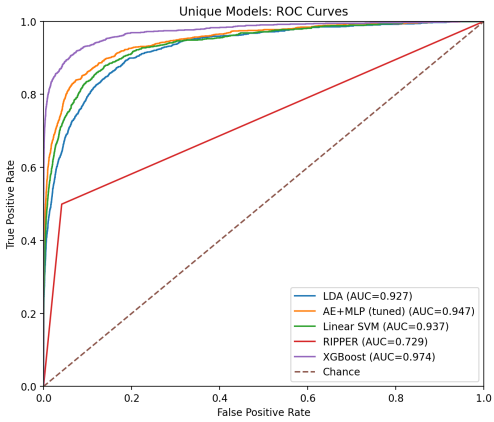
Comparison 1: Baseline Ensemble with Threshold-Aware Aggregation vs. other baseline ensemble aggregation methods: Figure 11, Figure 12, and Figure 13 compare the five baseline ensemble aggregation methods used by the group. My Threshold-Aware baseline sits at the recall-focused end of the spectrum: it achieves one of the strongest fraud recalls, but its precision and F1 are lower than the more conservative aggregations (e.g., those designed to reduce false positives) like Trimmed Mean and Weighted Average Aggregation.

Comparison 2: My unique model (Linear SVM) vs other unique models: Figure 25 compares the five unique models created by the group. In this comparison, Linear SVM sits in the middle: its recall is not the highest among the unique models, but it achieves a noticeably better balance than the weakest unique models (like LDA and RIPPER), especially when considering F1.The ROC plot also shows that Linear SVM has strong ranking performance overall.

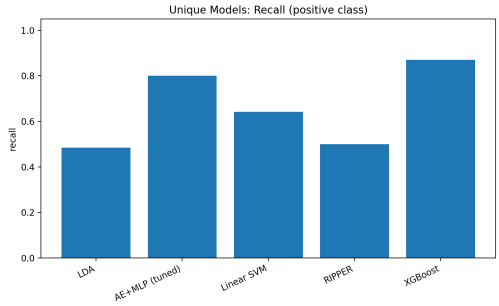
Comparison 3: Threshold-Aware Aggregation and Linear SVM vs. all other models: From my two models, the key finding is that "best performance" depends on what we mean by best. If the goal is maximum fraud recall (catch as many fraud cases as possible), my Threshold-Aware baseline is stronger than my Linear SVM (0.909 vs 0.641 recall). If the goal is a more balanced fraud detector (fewer false alarms while still catching fraud), my Linear SVM model performs better on precision, F1 and PR-AUC than the Threshold-Aware baseline.

Looking across the group's full set of models using Figure 11, Figure 13, Figure 12 and 25: Some ensemble baselines are clearly useful for pushing recall higher (they act like a "wide net" that catches more fraud but increases false positives).Some individual models show very strong overall ranking (ROC) and can achieve a better balance on precision/F1 than recall-first ensembles, depending on the model and threshold choice.

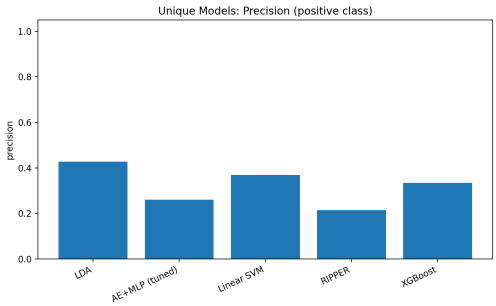
So, the practical conclusion is that Ensemble approaches tend to be the best option when recall is the top priority, while advanced models like XGBoost and SVM can be more competitive (or better) when we care about balanced performance and reducing false positives.



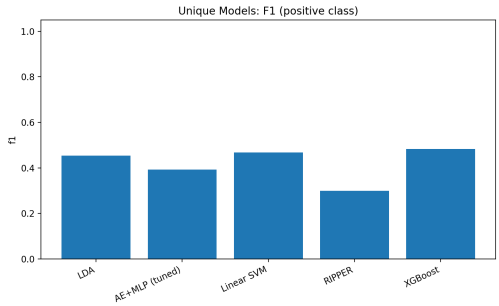
(a) ROC curves for unique models.



(b) Recall (fraud class) across unique models.



(c) Precision (fraud class) across unique models.



(d) F1-score (fraud class) across unique models.

Figure 25: Unique model comparison across the five individual models produced by the group.

Summary: In conclusion, my two models highlight the main trade-off in fraud detection on imbalanced data.**Threshold-Aware ensemble** is best when we want to miss as little fraud as possible, while **Linear SVM** gives a cleaner balance with fewer false positives. In a real fraud system, the choice between these approaches would depend on operational constraints (e.g., how many alerts a review team can handle) and the business cost of missed fraud vs false alarms.

Student 4: Amarjeet

This section presents a comprehensive evaluation of the models developed for fraud detection, specifically focusing on the Weighted Average Aggregation baseline ensemble and XGBoost classifier. The evaluation includes multiple metrics, including precision, recall, F1-score, ROC-AUC, and average precision to assess model performance across different dimensions.

4.1 Evaluation Metrics

The models were evaluated using standard binary classification metrics with particular emphasis on the minority class (fraud cases), given the severe class imbalance in the dataset. The key metrics include:

i) Precision (Positive Class): The proportion of predicted fraud cases that are actually fraudulent. High precision means fewer false alarms.

ii) Recall (Positive Class): The proportion of actual fraud cases correctly identified. High recall means fewer missed frauds.

iii) ROC-AUC: Measures how well the model discriminates between classes across all classification thresholds.

iv) Average Precision: Summarizes the precision-recall curve, especially useful for imbalanced datasets like ours.

4.2 Individual Model Performance

4.2.1 XGBoost Model

XGBoost achieved strong overall performance with 95.89% accuracy and 0.974 ROC-AUC. For the fraud class specifically:

i) Precision: 0.334 (33.4%)

ii) Recall: 0.870 (87.0%)

iii) F1-score: 0.483

iv) Average Precision: 0.820

The confusion matrix shows XGBoost caught 1,150 of the 1,322 fraud cases (87%), missing only 172. The downside was 2,289 false alarms, which explains the precision of 33.4%. This is the classic precision-recall trade-off: the model prioritizes catching fraud over avoiding false positives.

4.2.2 Weighted Average Aggregation Baseline

The Weighted Average baseline achieved 93.13% accuracy with 0.957 ROC-AUC. For fraud detection:

i) Precision: 0.218 (21.8%)

ii) Recall: 0.820 (82.0%)

iii) F1-score: 0.345

iv) Average Precision: 0.628

This baseline identified 1,084 of 1,322 fraud cases and missed 238. It produced 3,878 false positives more than XGBoost, with a precision of just 21.8%. While the 82% recall is good, the precision-recall balance is clearly worse than XGBoost.

4.3 Comparative Analysis

4.3.1 Comparison Against Other Unique Models

Figure 18 shows the precision-recall trade-off for all unique models plotted against the Trimmed Mean baseline. XGBoost is in a strong position with 0.87 recall and 0.334 precision, outperforming most other approaches.

Compared to the Trimmed Mean baseline, XGBoost achieves much higher recall (0.87 vs. 0.66) though with lower precision (0.334 vs. 0.514). The ROC-AUC advantage (0.974 vs. 0.96

from Figure 20) suggests XGBoost has better overall discrimination ability. Against LDA, XGBoost far exceeds it in recall (0.87 vs. 0.44) with similar precision, making it clearly the better fraud detector. Linear SVM does better than LDA but still falls short. XGBoost has a higher recall (0.87 vs. 0.67) with comparable precision (0.334 vs. 0.366), which means it catches more fraud without sacrificing much precision.

The comparison with AE+MLP (tuned) is closer since both are advanced methods. XGBoost still achieves higher recall (0.87 vs. 0.81) and better precision (0.334 vs. 0.263). RIPPER performs the worst overall and XGBoost beats it on both metrics with 0.87 vs. 0.56 recall and 0.334 vs. 0.22 precision.

Figure 21 shows the ROC curves more clearly. XGBoost has the highest AUC at 0.974, followed by AE+MLP at 0.947 and Linear SVM at 0.937. The gap between XGBoost's curve and the diagonal chance line is substantial, confirming strong discriminative power. RIPPER trails badly at 0.729 AUC, barely better than random guessing in some regions.

4.3.2 Comparison Against Other Baseline Ensembles

Figures 13 and 11 show how the five baseline methods compare. My Weighted Average sits in the middle with 0.82 recall which is better than Trimmed Mean (0.66) and Entropy Aggregation (0.80), but worse than Max Prob (0.92) and Threshold-Aware (0.91).

For precision, Weighted Average is quite low at 0.22. Most other methods have similar precision except Trimmed Mean, which gets 0.51 but pays for it with much lower recall. So there's a clear pattern here: most baselines go for high recall and accept low precision. Weighted Average follows this pattern too. In fraud detection, it makes sense to prioritize catching fraud cases over worrying about false alarms. Trimmed Mean is the outlier; it's more conservative and gets better precision but misses more fraud.

The choice of aggregation method clearly affects results. Weighted Average is a reasonable compromise, though a couple of my teammates' methods get higher recall if that's the priority.

4.3.3 Overall Comparison

Comparing both my models against all other models reveals several findings: XGBoost comes out on top with the best ROC-AUC (0.974) across all models for both unique and baseline. It has the best balance between catching fraud and avoiding false alarms. Among just the unique models, XGBoost also has the highest recall at 0.87. Max Prob and Threshold-Aware baselines get slightly higher recall (0.91-0.92), but their precision is much worse, so XGBoost is still the clear winner for unique models.

My Weighted Average baseline is more in the middle in terms of performance. It performs about the same as Entropy Aggregation but falls well short of XGBoost. Compared to the other baselines, it has decent recall and typical precision for this type of voting approach. The gap between XGBoost and Weighted Average is pretty substantial: XGBoost is 2.76% better on accuracy, 0.017 on ROC-AUC, 0.192 on average precision, and 0.116 on precision for fraud cases. This shows that gradient boosting works better than simple ensemble voting for this problem. From a practical standpoint, XGBoost's combination of 87% recall and 33.4% precision makes sense when missing fraud is more expensive than checking false alarms. If you need higher precision instead, Trimmed Mean gets 51% precision, but with a recall of just 66%.

XGBoost clearly outperforms both the Weighted Average baseline and most other models in our group. With the highest ROC-AUC (0.974) and best recall (0.87) among unique models, it's the strongest choice for fraud detection on this dataset. The Weighted Average baseline offers a simpler alternative with decent recall (0.82) but lower precision, which could be preferred if interpretability or speed matters more than maximum performance. Overall, the comparison confirms that advanced techniques like XGBoost provide real advantages over traditional ensemble methods for this fraud detection problem.

Student 5: Nneamaka

4.4 LDA Performance Evaluation

The LDA model achieved a ROC-AUC score of 0.927, PR-AUC score of 0.462, Fraud recall and precision scores being 0.485 and 0.427 respectively. The ROC curve shows that there is a clear difference between fraudulent and legitimate transactions, although the precision/recall curve shows that detecting rare fraudulent transactions while maintaining precision can be a bit difficult.
The confusion matrix showed that out of all fraudulent transactions in the test set, 641 were correctly identified as frauds (true negatives), and 681 were falsely identified as frauds (false negatives). Although the LDA model could capture a meaningful proportion of fraud, almost half of the fraudulent transactions were undetected. This shows that its ability to detect fraud is constrained by its linear assumptions and sensitivity to class imbalance.
The LDA model was also used to point out some key features that influence the increase and decrease in fraud likelihood. For example, the likelihood of a fraudulent transaction occurring could be influenced by high shipping distance, large transaction amounts, certain countries and merchant categories, and specific transaction channels. It could be reduced with successful AVS and CVV checks, presence of 3-D Secure, and older account age.
These results align well with real-world fraud risk patterns and provide actionable insight for risk teams.

4.5 Trimmed Mean Aggregation Ensemble Performance Evaluation

The Trimmed Mean Aggregation model achieved a ROC-AUC score of 0.96 PR-AUC score of 0.672, Fraud recall and precision scores being 0.658 and 0.514 respectively. In comparison to the LDA, it shows a significant improvement across all the fraud-focused metrics. The precision/recall curve in Figure 9 above shows that it is more stable and has higher precision across a wider range of recall values. This means that the ensemble provides more reliable probability estimates.

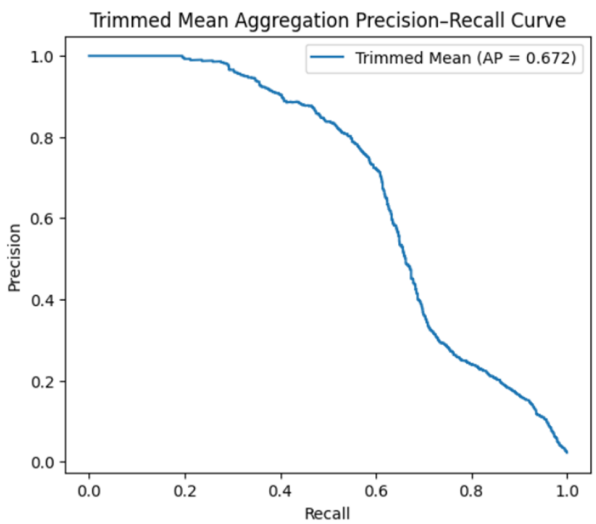


Figure 26: Precision/Recall Curve for Trimmed Mean Aggregation Ensemble.

Out of all the fraudulent transactions in the test set, the confusion matrix shows that the model correctly identified 870 frauds, and missed 452, which is much better than the result of the LDA model. This indicates a major reduction in the number of fraud cases that were not identified, while maintaining a high performance on legitimate transactions. This improvement means that more fraudulent transactions can be stopped before completion, customer experience

could be better due to more false positives, and there could be fewer losses due to undetected fraudulent transactions. Therefore, the baseline model with trimmed mean aggregation provides a more practical balance between fraud prevention and customer experience.

4.6 Comparison of Trimmed Mean Aggregation and LDA Models with Other Baseline models

In comparison with the other baseline models developed within the group:
i. The max probability and threshold aware aggregation achieved good recall scores of 0.9138 and 0.9092, and low precision scores of 0.1006 and 0.1058 respectively. Although the trimmed mean aggregation model has a lower recall score (0.6581), it reduced the number of false positives, which makes it practical for operation in a real business setting.
ii. Entropy based aggregation improved the fraud recall score (0.7972) in comparison to the LDA model fraud recall score, but produced a low precision score, meaning that it does not easily detect actual fraudulent transactions. This creates more workload for fraud analysts and reduces customer experience.
iii. Weighted Average Aggregation produced a stronger recall (0.8199) but lower precision (0.2185) than the trimmed mean aggregation model. This means that even though it improves the detection of fraudulent transactions, it produced much more false positives than the trimmed mean ensemble. The trimmed mean ensemble shows more balance between fraud detection and customer disruption.
Overall, the trimmed mean aggregation model provides the most operationally balanced trade-off in comparison to the other baseline ensemble strategies. It improves the detection of frauds more significantly than other models like the LDA and avoids extreme false positive rates such as those seen in some of the baseline ensemble strategies with very high recall scores. Figure 11 above shows the performance of the trimmed mean aggregation model in comparison to the other baseline ensembles based on the precision metric.

4.7 Comparison with other Advanced Models

In comparison with the more advanced models developed within the group:
i. The XGBoost model had the best performance on fraud detection which is shown in Figure 22 above. It obtained the highest ROC-AUC (0.974) and PR-AUC (0.82) scores compared to LDA and trimmed mean ensemble. This shows that it ranked fraud transactions most effectively, which is very valuable in imbalanced situations. It means that thresholds can be set based on business requirements and tolerance, and it will still rank transactions correctly. However, the model is more complex and not very interpretable.
ii. Linear SVM performed closely to the trimmed mean ensemble model but achieved a lower precision score (0.3685). This means that the SVM cannot detect real fraudulent transactions like the trimmed mean aggregation approach.
iii. The Threshold Tuned Autoencoder + MLP achieved a high recall (0.8003) and low precision score (0.2603) which shows that it is more sensitive to anomalies instead of targeted fraud detection. The trimmed mean ensemble produced less false positives than this model.
iv. The Ripper model did not perform as well as the LDA model and trimmed mean ensemble which shows that it is not good enough for the detecting fraudulent transactions.

Each of the models trained for fraud detection has its strengths and weaknesses. The trimmed mean ensemble tends to be simpler and more robust, while the LDA provides insight into the features that influence fraudulent activity. The results show that there is not a universally 'best' model for fraud detection. Instead, a good fraud system can consist of high-performance models for fraud risk scoring, models that are easily interpretable and robust for monitoring and good decision making. LDA is a model that is interpretable and transpar-

ent, which helps in explaining the drivers of fraud risks, but it is not enough for preventing frauds. Trimmed mean aggregation improves the performance of fraud detection and maintains an acceptable precision score, which makes it a strong baseline ensemble. It offers the best balance between less false alarms and customer experience. Advanced models like XGBoost perform very well and will be good for risk scoring. All in all, the best strategies for fraud detection combine detection strength with features such as stability, explainability, and business awareness. Figure 18 above shows the precision/recall trade-off between the unique models and the best baseline models. It demonstrates the trade-off between fraud capture and the quality of detection, supporting different operational strategies depending on business risk tolerance.

5 Discussion, Conclusion and Recommendations

5.1 Discussion

Fraud detection is a high-impact business problem because the target class is rare but costly. In our dataset, fraud represents approximately 2.21% of transactions (1,322 fraud cases out of 59,939 test transactions). This class imbalance means that overall accuracy can look strong even when a model misses a large number of fraud cases. Therefore, in line with the assessment brief, we focused our evaluation and interpretation on fraud-class precision, recall, F1-score, ROC–AUC, and PR–AUC, supported by confusion matrices to translate model performance into business outcomes (missed fraud vs. investigation workload).

5.1.1 What the results indicate about the business trade-off

Across all models, the central finding is that "best" performance depends on the operational goal:

- **Minimise missed fraud (high recall):** prioritise recall to reduce false negatives, which represent fraud that slips through.
- **Minimise unnecessary friction (high precision):** prioritise precision to reduce false positives, which represent legitimate customers being flagged or analysts investigating non-fraud.

This is the classic precision–recall trade-off: pushing recall upward typically increases false positives, while pushing precision upward can increase false negatives.

5.1.2 Baseline ensembles: aggregation strategy changes operational behaviour

A key contribution of this project is showing that *aggregation choice* in an ensemble can substantially change the business behaviour of a system, even when the same base models are used.

Recall-first baselines (wide-net screening). The most recall-oriented ensemble variants were Max Probability and Threshold-Aware aggregation. Max Probability achieved fraud recall = 0.914 (detecting 1,208 of 1,322 fraud cases, missing only 114), but at the cost of fraud precision = 0.101 and a very large false-positive workload (10,795 false alarms). Similarly, Threshold-Aware aggregation achieved fraud recall = 0.909 (1,202 detected; 120 missed) with fraud precision = 0.106 and 10,156 false positives. From a business perspective, these models are effective as an *early-stage screening layer* where the goal is to catch as much fraud as possible, but they require downstream capacity (manual review or a second-stage model) to manage alert volume.

Balanced baselines (decision-ready filtering). More conservative aggregation strategies delivered substantially better alert quality. Trimmed Mean aggregation was the strongest baseline overall by fraud F1-score, achieving precision = 0.514, recall = 0.658, F1 = 0.577, and PR–AUC = 0.672, while generating only 823 false positives. Weighted Average aggregation also performed well (precision = 0.218, recall = 0.820, F1 = 0.345, PR–AUC = 0.628), but still produced 3,878 false positives. Operationally, Trimmed Mean represents the best baseline "middle ground" for a business that wants a usable fraud detection system with manageable investigation workload.

5.1.3 Unique models: stronger ranking and better trade-offs

Among the five unique models, XGBoost delivered the strongest overall discrimination and alert quality. It achieved ROC–AUC = 0.974 and PR–AUC = 0.820, with fraud recall = 0.870 (1,150 detected; 172 missed) and fraud precision = 0.334 (2,289 false positives). This combination indicates that XGBoost is effective for *risk scoring and prioritisation*: it captures most fraud while keeping false positives far below recall-first baselines.

Other models illustrate additional trade-offs:

- **Linear SVM (threshold-tuned):** precision = 0.369, recall = 0.641, F1 = 0.468, PR–AUC = 0.538, representing a cleaner precision/recall balance than recall-first baselines.
- **Threshold-tuned Autoencoder+MLP:** recall = 0.800 with precision = 0.260 and PR–AUC = 0.605, making it suitable for analyst-driven review stages where recall remains important.
- **LDA:** precision = 0.427 with recall = 0.485 and PR–AUC = 0.462, representing a more conservative detector.
- **RIPPER:** precision = 0.214, recall = 0.499, F1 = 0.300, ROC–AUC = 0.729, PR–AUC = 0.118. RIPPER provides interpretability via explicit rules, but its detection and ranking performance are weaker than other approaches.

5.1.4 Why PR–AUC and confusion matrices mattered in this project

Because fraud is rare, PR–AUC (Average Precision) is particularly useful: it summarises how precision changes as recall increases and focuses directly on the quality of flagged alerts. In business terms, PR–AUC is closely aligned with operational KPIs such as "how many of the flagged transactions are truly fraud" at different catch rates. Confusion matrices were also essential because they translate performance into counts: false negatives correspond to missed fraud losses, while false positives correspond to investigation workload and potential customer friction.

5.1.5 Summary of the main insight

Overall, the group results confirm that fraud detection is not a single-metric optimisation problem. Recall-first models behave like "wide nets" and are best when missing fraud is unacceptable, while balanced models deliver higher precision and lower operational overhead. For decision support in a business environment, the recommended choice depends on the organisation's cost tolerance, review capacity, and customer experience constraints.

The discussion table is designed to *tell the story of trade-offs* clearly for business readers, rather than repeat the full results section. We therefore chose five "anchor" models that span the main operating points of the project:

Table 3: Key models illustrating the precision–recall trade-off (fraud class).
Models were selected to represent (i) highest-recall ensemble, (ii) alternative high-recall ensemble, (iii) best-balanced baseline, (iv) best-performing individual model, and (v) interpretable rule-based model.

Model	**Prec.**	**Rec.**	**F1**	**ROC–AUC**	**PR–AUC**	**FP**	**FN**
Max Probability (baseline)	0.101	0.914	0.181	0.937	0.288	10795	114
Threshold-Aware (baseline)	0.106	0.909	0.190	0.917	0.134	10156	120
Trimmed Mean (baseline)	0.514	0.658	0.577	0.960	0.672	823	452
XGBoost (unique)	0.334	0.870	0.483	0.974	0.820	2289	172
RIPPER (unique)	0.214	0.499	0.300	0.729	0.118	2420	662

- **Max Probability (baseline):** Included because it is the clearest example of a *recall-first* ensemble decision rule. Max aggregation behaves like a high-sensitivity alarm (if any base learner is confident, the ensemble flags the case), which tends to increase recall at the cost of more false positives—a direct manifestation of the precision–recall trade-off.
- **Threshold-Aware (baseline):** Included as an *alternative recall-first baseline* to show that even within ensemble baselines, the aggregation/decision logic can shift the operating point (alert volume and precision–recall behaviour). This supports the discussion that "baseline" is a family of design choices, not a single benchmark.
- **Trimmed Mean (baseline):** Included because it represents the *best-balanced baseline* in our experiments: a substantial improvement in alert quality (precision/F1) and PR–AUC while keeping false positives low. This gives a practical counterpoint to recall-first baselines and makes the trade-off business-readable (workload vs missed fraud).
- **XGBoost (unique):** Included as the strongest *individual supervised learner* in our results, with the best overall ranking and alert quality (highest PR–AUC). For imbalanced fraud detection, PR curves and Average Precision (PR–AUC) are particularly useful because they focus on performance on the positive class across thresholds.
- **RIPPER (unique):** Included to represent the *interpretability-first* end of the spectrum. RIPPER produces human-readable IF–THEN rules, which is valuable for governance and auditability, but it may underperform compared with more flexible models on complex patterns

In short, these five models provide a compact but complete narrative: two recall-heavy ensembles, one balanced baseline, the best-performing individual learner, and an interpretable rule-based approach. This allows the discussion to remain focused on business implications while still being faithful to the experimental findings.

5.2 Conclusion

The purpose of this project was to find out which supervised learning approach performs the best in detecting fraudulent transactions within an imbalanced e-commerce dataset. This study made use of a combination of interpretable and advanced machine learning models.
Combining multiple supervised learning and ensemble models showed that there can be a great improvement in fraud detection as long as the models are carefully designed and evaluated. Analysis showed that meaningful patterns can be generated from transaction and behavioural data to support the detection of fraudulent activities.

Generally, the models performed well in detecting fraudulent activity. This was proven using the appropriate evaluation metrics for imbalanced data such as Precision/Recall analysis and ROC-AUC. The results of this study showed that there is no single modelling approach that is universally optimal for the detection of frauds. Simpler and more interpretable models such as

the LDA provided valuable insight into the features that influence the increase and decrease if frauds. More complex approaches like the ensemble models (baseline), yielded stronger results in fraud detection. This points out the common trade-off in business analytics between model transparency and predictive power.

Evaluation not only showed the performance of the models but also disclosed important characteristics of the data. For example, the occurrence of frauds is not rare but random. It is associated with specific and unusual patterns such as very large transaction amounts, longer shipping distances, specific locations, and weaker security checks. The presence of skewed distributions and extreme data values showed the need for robust preprocessing, appropriate scaling, efficient models, and evaluation methods that provide more insight than accuracy. The project demonstrated how heterogeneous transaction data can be transformed into actionable and realistic insights.

This study concludes that there is no universally optimal model for fraud detection but a combination of robust models with different strengths and features. That is what makes a good fraud detection system.

5.3 Recommendations

Based on the results of this study, these are the recommendations proposed for businesses that want to improve fraud detection using data-driven approaches:

I. It is better to combine models rather than relying on a single one: The evaluation and analysis of the models' performance show that different models capture different aspects of fraudulent behaviour. The ensemble models and aggregation approach are diverse and provide a more stable and reliable performance.

II. Use Interpretable Models for Insight and Management: Although ensemble models perform much better in predictions, simpler and interpretable models should be valued as well because they explain why transactions are flagged as fraudulent and also help in decision making.

III. Use Suitable Evaluation Metrics for Fraud Detection: Due to the high class imbalance, accuracy alone is not sufficient enough in the task of fraud detection. Businesses need to prioritise other metrics including precision, recall, and ROC-AUC as they are better for imbalanced datasets and provide more insight into how well fraud cases are identified and if there are false alarms.

IV. Continuously Monitor and Update Models: It is important for models to be retrained regularly using recent data because fraud patterns keep changing over time. The performance of the models should be evaluated using the appropriate metrics that are well suited to imbalanced datasets. This makes sure that frauds detection systems stay effective as customer and fraud behaviours change.

V. Decision Thresholds Should Suit Business Priorities: Businesses need to adjust their thresholds based on how costly fraud is. Blocking genuine customers due to false fraud alarms can damage customer experience, while missing fraudulent transactions can lead to financial loss. Adjusting the decision threshold helps in controlling this trade-off. It ensures that fraud detection systems support both operational efficiency and customer satisfaction.

This multi-layer structure matches industry practice where rule-based systems remain common, and combining rules with AI/ML can simplify workflows and reduce the number of required rules while improving detection. This project demonstrates how useful data-driven analytics is when applied to transactional data in order to improve fraud detection, risk management, operational efficiency and customer experience.

6 Authors Contribution Statements

Student	Contribution Statement
Lohit	Implemented Baseline Model with Entropy-Based Confidence Weighting Aggregation and Autoencoder with MLP Classifier, report writing, and Research and Design of the baseline model.
Nneamaka	Implemented LDA and Baseline Model with Trimmed Mean aggregation, data preprocessing and report writing.
Shivam	Implemented Ripper and Baseline Model with Max Probability (Recall-Oriented) Aggregation, report writing, and Research and Design of the baseline model.
Amarjeet	Implemented XGBoost, Baseline Model with Weighted Average Aggregation,data preprocessing and report writing.
Upasana	Implemented SVM and Baseline Model with Threshold-Aware Aggregation, data preprocessing and report writing.

Group Meetings Logbook

Meeting No	Date	Discussion	Work Done	Attendees
1	02/11/2025	First meeting to review the coursework brief, deadlines, and possible domain areas	Reviewed assessment requirements and agreed to focus on business relevant problems	All members
2	18/11/2025	Explored and compared open-source datasets suitable for this coursework. Decided to work with the customer spending prediction. Agreed to send an email to the lecturer to confirm dataset.	Selected the online customer spending dataset and agreed on the problem statement and objectives	All members
3	04/12/2025	Changed the dataset and domain area to online fraud transactions. Planned the overall structure of the project. Finalised on what tools will be best for the project and divided responsibilities across the group.	Finalised on a dataset. Assigned roles for R preprocessing, baseline models, individual models, and report sections	All members
4	16/12/2025	Progress meeting. Preprocessing on the fraud transactions dataset was done	Completed data cleaning and preprocessing in R and exported the final dataset for modelling in Python	All members
5	30/12/2025	Reviewed modelling results and evaluation metrics. Discussed comparison of individual and ensemble models and business interpretation of findings.	Consolidated results, generated comparison plots, and started writing reports	All members

References

[1] Rok Blagus and Lara Lusa. Smote for high-dimensional class-imbalanced data. *BMC bioinformatics*, 14(1):106, 2013.

[2] Tianqi Chen. Xgboost: A scalable tree boosting system. *Cornell University*, 2016.

[3] William W Cohen et al. Fast effective rule induction. In *Proceedings of the twelfth international conference on machine learning*, pages 115–123, 1995.

[4] Ian Goodfellow. Deep learning, 2016.

[5] International Journal of Advanced Computer Science and Applications. Enhancing threat detection in financial cyber security through auto encoder-mlp hybrid models. *International Journal of Advanced Computer Science and Applications*, 15(4), 2024.

[6] Geoffrey J McLachlan. *Discriminant analysis and statistical pattern recognition.* John Wiley & Sons, 2005.

[7] Md. Arif Istiek Neloy, Nazmun Nahar, Mohammad Shahadat Hossain, and Karl Andersson. A weighted average ensemble technique to predict heart disease. In *Proceedings of the Third International Conference on Trends in Computational and Cognitive Engineering*, pages 17–29. Springer Nature, 2022.

[8] Umut Tüygür. E-commerce fraud detection dataset, 2024.

[9] Tuan L Vo, Uyen Dang, and Thu Nguyen. Directly handling missing data in linear discriminant analysis for enhancing classification accuracy and interpretability. *arXiv preprint arXiv:2407.00710*, 2024.

[10] De Zhang. A sample-level entropy-weighted stacking method for diabetes prediction. *Journal of Technology Innovation and Engineering*, 1(4), 2025.